



Bilkent University
Department of Computer Engineering

Senior Design Project
Team T2507

SCUBAMIND - MENTAL HEALTH

Detailed Design Report

22103231 Merve Güleç merve.gulec@ug.bilkent.edu.tr

22103775 Metin Çalışkan metin.caliskan@ug.bilkent.edu.tr

22101998 Muhammed Fatih Başal fatih.basal@ug.bilkent.edu.tr

22103416 Murathan Işık murathan.isik@ug.bilkent.edu.tr

22103406 Yiğit Koşum yigit.kosum@ug.bilkent.edu.tr

Hamdi Dibeklioglu
İlker Burak Kurt and Mert Bıçakçı

12.03.2026

Contents

1. Introduction.....	4
1.1 Purpose of the System.....	4
1.2 Design Goals.....	5
1.3 Definitions, Acronyms and Abbreviations.....	6
Definitions of Key Terms.....	6
Acronyms and Abbreviations.....	8
1.4 Overview.....	9
2. Current Software Architecture.....	9
3. Proposed Software Architecture.....	11
3.1 System Overview.....	11
3.2 Subsystem Decomposition.....	12
3.2.1. Flutter Module.....	12
3.2.2 Native Android Layer.....	13
3.2.3 AI Module.....	13
3.2.4 LLM Companion.....	14
3.3 Hardware / Software Mapping.....	14
3.3.1 Component Mapping Table.....	15
3.3.2 Hardware and Software Interaction.....	15
3.4 Persistent Data Management.....	16
3.4.1 Database Architecture and Security.....	16
3.4.2 Data Models and Entities.....	16
3.4.3 Schema Evolution and Migrations.....	17
3.4.4 Data Lifecycle and Sovereignty.....	17
3.5 Access Control and Security.....	17
3.5.1 Multi-Layered Data Security.....	17
3.5.2 Authentication and Authorization.....	17
3.5.3 Data Privacy and Sovereignty.....	18
4. Subsystem Services.....	18
4.1 Mobile App Subsystem.....	18
4.2 Backend Subsystem.....	20
4.3 ML / Analytics Subsystem.....	22
4.4 Method Level Design.....	23
4.4.1 Core Algorithms.....	23
4.4.2 Sequence Diagrams.....	24
4.4.3 Activity Diagrams.....	25
4.4.4 Class Diagram.....	26
4.4.5 Use Case Diagram.....	27
4.4.6 Data Flow Diagrams.....	28
4.5. Development Methodology.....	29
4.5.1 Development Model.....	29
4.5.2 Development Phases.....	30
5. Test Cases.....	32
5.1 Functional Test Cases.....	32
5.1.1 TC-F01 — Home page loads with mental health overview.....	32
5.1.2 TC-F02 — Pull-to-refresh reloads all Home blocs Test.....	33
5.1.3 TC-F03 — Voice recordings list loads from DB.....	33

5.1.4 TC-F04 — Play/Pause toggles correctly.....	33
5.1.5 TC-F05 — Confirm delete removes recording.....	34
5.1.7 TC-F06 —SpeechToTextModel loads encoder, decoder, and tokenizer.....	34
5.1.7 TC-F07 —Inference returns empty string for silent audio.....	35
5.1.8 TC-F08 — Chat page sends and displays messages.....	35
5.1.9 TC-F09 — PHQ questionnaire loads and submits.....	35
5.1.10 TC-F10 — Onboarding flow completes and is not shown again.....	36
5.1.11 TC-F11 —Interactive elements have adequate touch target.....	36
5.1.12 TF-F12 —Transcribing shown while transcription is null.....	36
5.1.13 TC-F13 —Inference fails gracefully when model not loaded.....	37
5.1.14 TC-F14 —Chat page loads and displays message list.....	37
5.1.15 TC-F15 —Empty message is not sent.....	37
5.2 Non-Functional Test Cases.....	38
5.2.1 TC-P01 — ASR inference completes within 10 seconds.....	38
5.2.2 TC-P02 — Voice recordings list loads within 2 seconds for 50+ entries.....	38
5.2.3 TC-P03 — Home page initial load completes within 3 seconds.....	39
5.2.4 TC-S01 — Voice recordings stored in private app directory.....	39
5.2.5 TC-S02 — Unauthorized DB access attempt is blocked.....	39
5.2.6 TC-S03 — PHQ data not stored in plain-text shared preferences.....	40
5.2.7 TC-UX01 — User can start a recording without instructions.....	40
5.2.8 TC-UX02 — Error messages are clear and actionable.....	40
5.2.9 TC-UX03 — Interactive elements have adequate touch targets.....	41
5.2.10 TC-R01 — System works under heavy load (concurrent transcriptions).....	41
5.2.11 TC-R02 — App recovers from ASR crash without data loss.....	41
5.2.12 TC-R03 — Data persists after unexpected app termination.....	42
6. Consideration of Various Factors in Engineering Design.....	42
6.1 Constraints.....	42
6.1.1 Public Health.....	42
6.1.2 Public Safety.....	43
6.1.3 Public Welfare.....	43
6.1.4 Global Factors.....	43
6.1.5 Cultural Factors.....	44
6.1.6 Social Factors.....	44
6.1.7 Environmental Factors.....	44
6.1.8 Economic Factors.....	44
6.2 Standards.....	46
7. Teamwork Details.....	47
7.1 Contributing and functioning effectively.....	48
7.2 Collaborative environment.....	48
7.3 Leadership.....	49
8. Glossary.....	50
9. References.....	50

1. Introduction

Mental health disorders, particularly depression, represent a major global health challenge affecting millions of individuals worldwide. According to the World Health Organization, depression is one of the leading causes of disability globally and significantly affects individuals' daily functioning and quality of life [1].

Traditional mental health monitoring methods typically rely on periodic clinical assessments and self-reported questionnaires. While these approaches provide useful information, they often fail to capture the continuous and dynamic behavioral changes occurring in everyday life. As a result, important behavioral patterns related to mental well-being may remain undetected between clinical visits [2].

Recent advances in mobile sensing technologies and artificial intelligence have enabled new approaches for continuous behavioral monitoring. Modern smartphones and wearable devices contain various sensors capable of capturing behavioral and physiological signals such as mobility patterns, smartphone usage behavior, sleep indicators, and heart rate variability. These signals can provide valuable insights into behavioral patterns associated with mental health conditions.

One emerging concept in this area is digital phenotyping, which refers to the continuous measurement of human behavior through data collected from personal digital devices [3]. By analyzing passive behavioral signals collected from smartphones and wearable devices, digital phenotyping enables continuous monitoring of behavioral patterns that may be associated with mental health risks.

ScubaMind is designed as a privacy-preserving mobile mental health monitoring system that leverages digital phenotyping techniques. The system collects behavioral and physiological signals from smartphone sensors and wearable devices and analyzes these signals using embedded artificial intelligence models. A key design principle of the system is its local-first architecture, where sensitive behavioral data is processed directly on the user's device to ensure strong privacy protection.

It is important to emphasize that ScubaMind is not intended to function as a clinical diagnostic tool. Instead, it acts as a decision-support and awareness system that helps users monitor behavioral trends over time and encourages them to seek professional support when necessary.

This report presents the detailed design of the ScubaMind system, including its architecture, subsystem structure, data management strategies, and testing procedures.

1.1 Purpose of the System

The primary purpose of the ScubaMind system is to provide a continuous and privacy-preserving platform for monitoring behavioral patterns related to mental well-being. Unlike traditional monitoring approaches that depend on sporadic clinical assessments or manual mood tracking, ScubaMind aims to capture behavioral signals passively and analyze them over time.

The system collects multimodal data from smartphone sensors and wearable devices, including behavioral signals such as smartphone usage patterns, mobility behavior, sleep indicators, and physiological signals. These signals are analyzed using embedded artificial intelligence models that identify patterns potentially associated with changes in mental well-being.

All behavioral analysis processes are performed directly on the user's device, ensuring that sensitive personal data remains private and under the user's control. Based on the analysis of behavioral signals, the system generates behavioral insights, trend visualizations, and a personalized mental health risk score.

In addition to passive monitoring, ScubaMind provides supportive features such as behavioral trend visualization and personalized recommendations designed to increase user awareness of daily habits and behavioral changes. By combining passive sensing, artificial intelligence, and privacy-preserving system design, ScubaMind aims to promote early awareness of potential mental health risks and encourage users to seek professional support when necessary.

1.2 Design Goals

The design of the ScubaMind system is guided by several key software engineering principles that ensure the system remains efficient, reliable, and user-centric.

Usability

- The system must provide an intuitive and user-friendly interface that allows users to easily interact with the application.
- Behavioral insights should be presented through clear dashboards, graphs, and visual summaries that are easy to interpret.
- The onboarding process should guide users step-by-step through permission settings and system configuration.
- Basic interactions should be achievable with minimal user effort.

Performance

- The system should process behavioral data efficiently while minimizing battery consumption and computational overhead on the device.
- Machine learning models should be optimized for mobile environments to ensure low-latency analysis results.
- User interface interactions should respond smoothly without noticeable delays.

Reliability

- Background services responsible for sensor data collection must operate continuously without interrupting normal device usage.
- The system should gracefully handle missing or incomplete sensor data and continue operating using available signals.

- Error-handling mechanisms should prevent corrupted data from affecting the analysis pipeline.

Security

- All collected user data must be encrypted and securely stored on the device.
- Access to sensitive behavioral data should be controlled through permission-based mechanisms.
- The system should follow privacy-preserving design principles to ensure user data protection.

Scalability

- The system architecture should allow the integration of new sensing modalities, machine learning models, or analytical modules in the future.
- The platform should support future feature expansions without requiring major architectural changes.

Maintainability

- The software architecture should follow modular and well-documented design principles to facilitate debugging and future development.
- Clear interfaces between system modules should simplify maintenance and code updates.

Modularity

- The system is designed as a layered architecture consisting of independent components such as the UI layer, native data collection layer, AI analysis module, and optional LLM companion module.
- Each subsystem operates independently while interacting through clearly defined interfaces.

1.3 Definitions, Acronyms and Abbreviations

Definitions of Key Terms

- **Behavioral Baseline:** A user-specific reference profile constructed from historical behavioral data and used to interpret future deviations in the user's daily patterns.
- **Behavioral Insight:** A system-generated interpretation of collected behavioral and physiological data, presented to the user to improve self-awareness rather than provide a clinical diagnosis.
- **Decision-Support System:** A software system designed to assist users in understanding patterns and making informed decisions, without replacing professional clinical judgment. In ScubaMind, the system functions as a decision-support and awareness tool rather than a diagnostic application.

- **Digital Phenotyping:** The passive and continuous collection of behavioral and physiological signals from personal digital devices, such as smartphones and wearables, in order to analyze patterns related to mental well-being. This concept forms the core methodological basis of ScubaMind.
- **Embedded AI Core:** The on-device machine learning component of ScubaMind responsible for analyzing multimodal signals and generating user-specific outputs such as mood predictions and risk scores.
- **Insight Dashboard:** The user-facing interface component that presents behavioral summaries, visual trends, and mental health risk indicators through understandable graphs and summary cards.
- **Local-First Architecture:** A privacy-preserving design approach in which sensitive user data is collected, stored, and processed primarily on the user's own device instead of external cloud servers. This is one of the fundamental architectural principles of ScubaMind.
- **Mood Prediction:** A system output that categorizes the user's current mental state into interpretable labels such as *Energetic*, *Low*, *Stressed*, or *Balanced*, based on behavioral and multimodal analysis.
- **Multimodal Fusion:** The process of combining multiple sources of behavioral and physiological data, such as phone usage, mobility, sleep, heart rate, and optional text or voice input, into a single analytical model to generate a meaningful risk assessment.
- **On-Device Inference:** The execution of trained machine learning models directly on the mobile device, allowing the system to produce predictions and analyses without sending raw data to the cloud.
- **Permission Control:** The mechanism through which users can manage data collection permissions, enable or disable specific sensing modalities, and control how their data is used and stored.
- **Personalized Recommendation:** A supportive action or suggestion generated by the system based on the user's recent behavioral patterns, such as activity reminders, breathing exercises, or habit-based recommendations.
- **Privacy by Design:** A system design principle that integrates privacy protection into the architecture from the beginning by minimizing collected data, enforcing secure storage, and giving users control over their data.
- **Risk Score:** A continuous or categorized score generated by the system to indicate potential changes in mental well-being based on multimodal behavioral analysis. In ScubaMind, this output is framed as a behavioral risk insight rather than a diagnosis.
- **Room Database:** The local database used in the ScubaMind system to securely store behavioral logs, user configurations, and generated analytical results on the device.
- **Wearable Integration:** The connection of external wearable devices to the mobile application in order to collect additional physiological signals such as heart rate, HRV, sleep duration, and activity information.

Acronyms and Abbreviations

- **AI (Artificial Intelligence):** A branch of computer science that enables machines to perform tasks requiring human-like intelligence, such as pattern recognition, classification, and prediction.
- **API (Application Programming Interface):** A set of protocols and interfaces that allows different software components or external services to communicate with one another.
- **DB (Database):** An organized collection of structured data that can be stored, managed, and retrieved efficiently.
- **FHIR (Fast Healthcare Interoperability Resources):** A healthcare data standard used for representing and exchanging electronic health information in a structured and interoperable way. ScubaMind considers this standard for future-proof health data representation.
- **Flutter:** A cross-platform UI development framework used in the ScubaMind project to implement the user interface layer of the application.
- **GPS (Global Positioning System):** A positioning technology used to derive mobility-related behavioral indicators such as movement intensity and routine variation.
- **HL7 (Health Level Seven):** A family of international standards for the exchange and integration of electronic health information.
- **HR (Heart Rate):** The number of heart beats per minute, used as a physiological indicator in behavioral and mental health analysis.
- **HRV (Heart Rate Variability):** The variation in time between consecutive heartbeats, often used as an indicator related to stress, recovery, and autonomic nervous system balance.
- **KVKK:** The Turkish Personal Data Protection Law governing the processing, storage, and protection of personal data in Türkiye. ScubaMind considers compliance with KVKK due to the sensitivity of mental health-related data.
- **LLM (Large Language Model):** A language model capable of generating and understanding human language. In ScubaMind, the LLM Companion is an optional module that provides supportive conversational guidance.
- **ML (Machine Learning):** A subfield of artificial intelligence focused on developing models that learn patterns from data and make predictions or decisions without being explicitly programmed for each case.
- **NLP (Natural Language Processing):** A field of AI concerned with analyzing and understanding human language, used in ScubaMind for transcript-based and language-based behavioral interpretation.
- **NPU (Neural Processing Unit):** Specialized mobile hardware designed to accelerate machine learning inference on devices.
- **OOM (Out of Memory):** A runtime condition in which available memory is insufficient for executing a computational task, especially relevant during model training or inference.
- **PDF (Portable Document Format):** A document format used by ScubaMind to generate shareable user reports summarizing trends and insights.
- **PHQ-9 (Patient Health Questionnaire-9):** A widely used nine-item depression questionnaire. In ScubaMind, PHQ-9 serves as a conceptual reference point for risk-aligned reporting rather than as a direct diagnostic output.

- **UI (User Interface):** The visual and interactive layer through which the user interacts with the application.
- **UX (User Experience):** The overall quality of the user's interaction with the application, including usability, accessibility, clarity, and comfort.
- **WHO (World Health Organization):** The international public health organization whose reports and definitions are used as key references in framing the mental health problem domain.

1.4 Overview

ScubaMind is designed as a privacy-preserving mobile system that monitors behavioral patterns related to mental well-being using passive sensing technologies and artificial intelligence. The system collects behavioral and physiological signals from smartphone sensors and wearable devices and analyzes these signals to identify patterns that may be associated with changes in mental well-being. The overall architecture of ScubaMind consists of several interconnected components, including the mobile application layer, native data collection services, and an AI-based analytics module that performs multimodal behavioral analysis and generates behavioral insights and risk indicators. The system follows a local-first architecture in which sensitive user data is primarily processed and stored on the user's device in order to ensure strong privacy protection.

This report presents the detailed design of the ScubaMind system. It explains the architectural structure of the application, the decomposition of the system into subsystems, and the interaction between system components. The report also describes the method-level design of the core algorithms, system diagrams, and data flow structures used within the system. In addition, the testing procedures designed to validate both functional and non-functional requirements of the application are discussed. Engineering considerations such as system constraints, relevant standards, and external factors affecting the design are also addressed. Finally, the report documents the teamwork process and collaboration practices followed during the development of the project.

2. Current Software Architecture

The rapid development of mobile technologies, wearable devices, and artificial intelligence has led to the emergence of many digital systems designed to monitor human behavior and support mental health assessment. These systems aim to provide insights about users' emotional states, behavioral patterns, and wellness indicators by collecting data from mobile devices or wearable sensors. However, existing solutions differ significantly in terms of sensing mechanisms, system architectures, and privacy considerations.

Below is an overview of several existing systems and platforms that are relevant to behavioral monitoring and digital mental health analysis.

- **StudentLife** is a research project developed at Dartmouth College that collects smartphone sensor data such as mobility patterns, phone usage, and activity levels to study behavioral changes related to academic performance and mental health [4]. The system

utilizes passive sensing to monitor daily behavioral patterns and applies machine learning techniques to analyze correlations between behavioral signals and mental well-being.

Differentiation: While the StudentLife system focuses primarily on behavioral sensing using smartphone data for research purposes, ScubaMind extends this approach by integrating both smartphone and wearable sensor data and embedding analysis directly on the user's device.

- **Moodpath** is a mobile mental health application that enables users to monitor their emotional well-being through periodic questionnaires and mood tracking features [5]. The application encourages users to regularly record their emotional states and provides feedback based on questionnaire results.

Differentiation: Moodpath relies mainly on self-reported user input, whereas ScubaMind incorporates passive behavioral sensing, allowing the system to analyze behavioral patterns without requiring constant manual interaction from the user.

- **Mindstrong Health** is a digital mental health platform that analyzes smartphone interaction patterns such as typing behavior and screen usage to detect cognitive and behavioral signals related to mental health conditions [6]. The system applies machine learning models to identify behavioral markers associated with mental health disorders.

Differentiation: Mindstrong focuses mainly on interaction-based behavioral signals, while ScubaMind combines multiple sensing modalities, including mobility data, wearable physiological signals, and smartphone usage patterns to achieve a more comprehensive behavioral analysis.

- **Apple Health / Google Fit** platforms collect physiological and activity-related data from wearable devices and smartphones, including heart rate, sleep duration, and physical activity levels [7]. These systems primarily focus on fitness tracking and general health monitoring.

Differentiation: Although these platforms provide valuable physical health metrics, they do not focus specifically on behavioral indicators related to mental well-being. ScubaMind extends beyond fitness tracking by applying artificial intelligence models to behavioral and physiological signals in order to detect potential mental health trends.

In summary, existing digital health systems typically rely on self-reported mood tracking, single-modal sensing, or cloud-based analytics architectures. These approaches may lead to limitations such as incomplete data collection, lack of behavioral context, or privacy concerns related to transmitting sensitive user data to remote servers. ScubaMind addresses these challenges by combining multimodal behavioral sensing, embedded artificial intelligence models, and a privacy-preserving local-first architecture, where sensitive behavioral data is processed primarily on the user's device.

3. Proposed Software Architecture

3.1 System Overview

The ScubaMind system is built using a layered and modular mobile software architecture designed to support continuous behavioral monitoring, on-device artificial intelligence analysis, and privacy-preserving data handling. The architecture separates user interaction, data collection, analytical processing, and optional conversational support into distinct subsystems in order to improve maintainability, extensibility, and system clarity. This layered structure also allows each subsystem to evolve independently without disrupting the rest of the application. The overall architecture of the proposed system is illustrated in Figure 1.

At the top of the architecture, the Flutter Module (UI Layer) provides the main user-facing interface of the system. This layer includes screens such as the Insight Dashboard, AI Chat Interface, onboarding and settings pages, and permission control components. It is responsible for presenting behavioral trends, risk summaries, and recommendations in an understandable and user-friendly format while also allowing the user to manage privacy settings and data permissions.

Beneath the UI layer, the Native Android Layer (Data & Storage) acts as the operational backbone of the system. This subsystem manages background services for collecting smartphone sensor data, usage statistics, wearable-derived physiological signals, and optional audio-based inputs. It also handles secure local persistence through the on-device database and user configuration storage. This layer ensures that data collection remains consistent with user preferences and that sensitive raw data is stored and managed locally.

The AI Module (Embedded Core) is the central analytical component of ScubaMind. It receives behavioral and physiological signals collected by the native layer and performs multimodal analysis using embedded machine learning models. This module combines digital usage patterns, mobility-related indicators, and wearable-based physiological signals to generate outputs such as behavioral summaries, mood-related insights, and a personalized risk score. Since inference is performed on-device, the system reduces privacy risks associated with transmitting sensitive behavioral data to external servers.

The LLM Companion is designed as an optional support module that provides a conversational interface for user guidance and reflective interaction. Unlike the AI analysis core, this component does not perform the primary risk estimation function of the system. Instead, it complements the system by delivering supportive communication and awareness-oriented interaction when enabled by the user. This separation ensures that the core behavioral analysis pipeline remains functional even when the LLM-based support component is not active.

A key architectural principle of ScubaMind is its local-first design. Sensitive behavioral and physiological data are collected, stored, and analyzed primarily on the user's device. This approach strengthens privacy protection, improves user trust, and supports offline or low-connectivity operation for core system functions. Overall, the proposed software architecture provides a cohesive structure that supports passive sensing, embedded

analysis, user-centered visualization, and privacy-aware system behavior in a unified mobile platform.

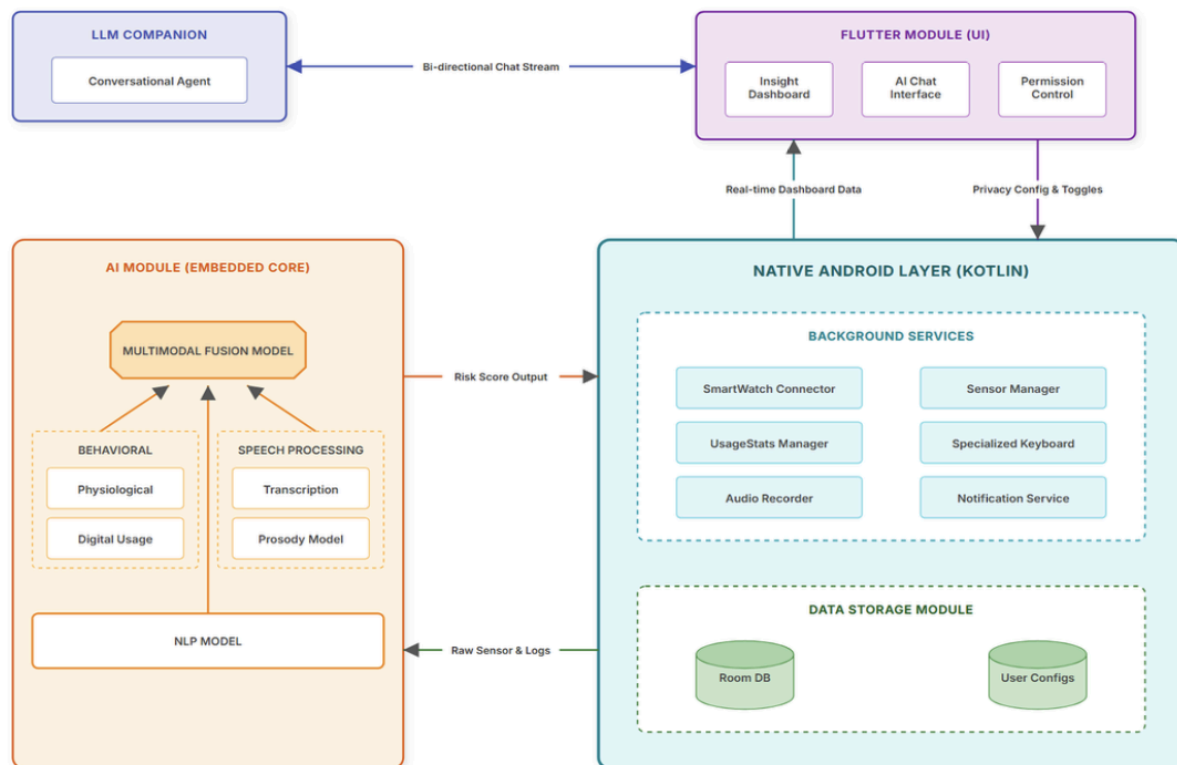


Figure 1. High-level architecture of the ScubaMind system showing the main subsystems and data flow.

3.2 Subsystem Decomposition

The ScubaMind system is architected into four primary, loosely coupled subsystems. This modularity ensures that heavy computational tasks (AI inference) do not block the user interface and that background data collection remains resilient to OS-level process management.

3.2.1. Flutter Module

Purpose: To serve as the primary gateway for user interaction, providing a transparent and intuitive interface for mental health monitoring and data sovereignty.

Responsibilities:

- **Data Visualization:** Transforming complex longitudinal datasets into human-readable charts on the Insight Dashboard.
- **Privacy Management:** Providing a centralized hub for the user to grant, revoke, or audit sensor permissions.
- **User Onboarding:** Guiding the user through the initial setup, including personal baseline establishment and wearable synchronization.

- Session Management: Handling the state of the conversational interface during interactions with the LLM Companion.

Data Flow:

- Inbound: Receives processed Risk Scores, Mood Predictions, and Behavioral Insights from the AI Module via the Native Layer.
- Outbound: Sends user-configured settings, manual mood entries, and chat transcripts to the Native Layer for local storage and further analysis.

3.2.2 Native Android Layer

Purpose: To act as the low-level operational backbone of ScubaMind, managing continuous, energy-efficient data acquisition and ensuring secure local persistence.

Responsibilities:

- Background Service Orchestrator: Manages the lifecycle of "Foreground Services" through persistent notifications to prevent OS-level termination. It acts as a Service Manager by monitoring SharedPreferences for configuration changes initiated by the Flutter UI. When a user toggles a sensor (e.g., GPS or IME) in the settings, the orchestrator dynamically starts or stops the corresponding background service based on these persistent flags.
- Passive Sensor Hub:
 - Mobility Manager: Polling GPS coordinates and Accelerometer data to calculate spatial variance and the user's "Radius of Gyration."
 - UsageStats Manager: Utilizing the Android UsageStatsManager API to monitor digital behavior patterns, including Screen On/Off events and App Usage Statistics.
 - Specialized Input Method (IME): A custom keyboard component that captures raw text input across various apps to provide data for linguistic analysis.
- Health Connect Bridge: Synchronizing physiological metrics from wearable devices, including Heart Rate Variability (HRV), Sleep Stages (Deep, REM, Light), and Resting Heart Rate.
- Encrypted Persistence Manager: Built on the Room Persistence Library, it manages AES-256 encrypted SQLite storage. It features a "Data Purge" trigger to facilitate the "Right to be Forgotten."

Data Flow:

- Inbound: Captures raw telemetry from phone sensors, system-level usage logs, raw text from the IME, and external wearable APIs. It also listens for Control Signals (Enable/Disable flags) stored in SharedPreferences from the Flutter Module.
- Outbound: Streams batched raw data and text fragments to the AI Module for inference and supplies the Flutter Module with historical data and real-time service status.

3.2.3 AI Module

- Purpose: To transform raw behavioral, linguistic, and physiological signals into actionable mental health insights through privacy-preserving, on-device machine learning.
- Responsibilities:
 - Digital Phenotyping Engine: Processing raw mobility traces from location_samples into behavioral biomarkers such as "Routine Index."
 - Linguistic & NLP Unit: * Textual Analysis: Analyzing raw text from the IME for sentiment, emotional valence, and linguistic complexity.
 - Prosody Analysis: Extracting acoustic features from voice_recordings, such as pitch variance, speech rate, and intonation, which serve as vocal biomarkers for emotional state.
 - Multimodal Fusion: Executing a late-fusion neural network that synthesizes phone behavior, text analysis, prosody features, and wearable physiological data into a unified Depression Risk Score.
 - On-Device Inference: Utilizing TensorFlow Lite to ensure all diagnostic-heavy analysis (including audio processing) is conducted locally.
- Data Flow:
 - Inbound: Consumes raw sensor batches, raw text sequences, and audio files fetched via the VoiceRecordingDao and UnifiedDataDao.
 - Outbound: Returns high-level analytical results (Risk Scores and Mood Categories) to the database and UI.

3.2.4 LLM Companion

- Purpose: To act as a "Psychological Assistant" by integrating the LLM API to facilitate supportive dialogue and encourage reflective Audio Journaling.
- Responsibilities:
 - Contextual Prompt Builder: Dynamically constructs prompts for Gemini by aggregating latest insights (e.g., "User has had < 5 hours of sleep for 3 days; mobility is low") to ensure highly personalized interaction.
 - Integration Layer: Orchestrates the API calls to LLM, leveraging its advanced reasoning to provide reflective listening and "diary-style" guidance.
 - Psychological Assistant Prompts: Specifically instructs the model to guide the user through a "daily diary" session, facilitating the collection of high-quality audio for prosody analysis.
 - Safety & Guardrail Layer: Uses a combination of LLM's built-in safety filters and custom keywords to monitor for high-risk intent (e.g., self-harm), triggering emergency protocols if necessary.
- Data Flow:
 - Inbound: Receives behavioral context, textual context from the client.
 - Outbound: Sends empathetic dialogue responses to the Flutter UI and logs conversation sentiment for the AI Module.

3.3 Hardware / Software Mapping

The ScubaMind system utilizes a hybrid mapping strategy. All sensitive data acquisition, pre-processing, and behavioral analysis are mapped to the user's local smartphone and Samsung wearable hardware. Only the high-level conversational logic is mapped to the cloud to leverage Large Language Models.

3.3.1 Component Mapping Table

Subsystem	Hardware Component	Software Stack / Technology
Flutter Module (UI)	Smartphone Display & CPU	Flutter SDK (Dart)
Passive Sensing	Smartphone Accelerometer, Counter GPS, Step	Android SDK (Kotlin), UsageStats API
Wearable	Samsung Galaxy Watch (Wear OS)	Samsung Health / Health Connect API
AI Module (Inference)	Smartphone NPU / GPU	TensorFlow Lite (TFLite)
Data Persistence	Smartphone Flash Storage	Room DB, SQLCipher (AES-256)
Conversational Agent	Cloud Infrastructure	Gemini API (REST / WebSocket)

Table 1 - Component Mapping Table

3.3.2 Hardware and Software Interaction

The deployment strategy ensures that sensitive data remains within the hardware boundary of the smartphone and the Samsung ecosystem:

- **Smartphone (On-Device):** This is the central node of the system. The Native Android Layer interacts directly with physical sensors, including the GPS for mobility, the

Accelerometer for movement intensity, and the Internal Step Counter for physical activity tracking. The AI Module maps directly to the phone's local processing units (NPU/GPU) using TensorFlow Lite to run the late-fusion neural network, ensuring that audio prosody and linguistic analysis are completed without external data transmission.

- **Samsung Wearable Integration:** Samsung Galaxy Watch hardware acts as a critical secondary sensor node. The software mapping utilizes the Health Connect Bridge to ingest physiological data—specifically Heart Rate Variability (HRV) and Sleep Stages—recorded by the Samsung Health sensors. This data is synchronized via Bluetooth LE and stored in the phone's local database for multimodal analysis.
- **Cloud (External API):** The LLM Companion is the only component mapped to an external cloud service. It functions as a lightweight client that sends anonymized behavioral context to the LLM API and receives empathetic dialogue responses.

3.4 Persistent Data Management

ScubaMind utilizes a Local-First persistence strategy to ensure user privacy and data sovereignty. All raw and processed data are stored within the smartphone's internal storage using a highly secure, encrypted relational database.

3.4.1 Database Architecture and Security

The system's storage backbone is built on the Room Persistence Library, serving as an abstraction layer over SQLite. To meet the sensitive nature of mental health data, the database is fortified with SQLCipher, providing full AES-256 page-level encryption.

- **Encryption Key Management:** The database passphrase is not hardcoded. It is dynamically generated as a UUID and stored in EncryptedSharedPreferences. This layer is mapped to the Android Keystore System, ensuring the master key is protected by hardware-backed security (TEE/StrongBox).
- **Initialization:** Upon application startup, the AppDatabase singleton is initialized using the SupportOpenHelperFactory to inject the encryption passphrase before any data access occurs.

3.4.2 Data Models and Entities

The database schema is organized into specialized entities to handle multimodal data streams efficiently. The primary entities include:

- **UnifiedDataEntity:** The core analytical table that consolidates daily metrics, including step counts, sleep duration, PHQ-9 scores, and raw keyboard text fragments.
- **LocationSampleEntity & MobilityDailyMetricsEntity:** These handle high-frequency GPS coordinates and the derived spatial biomarkers (e.g., Radius of Gyration, home coordinates).
- **VoiceRecordingEntity:** Manages metadata for local audio files, including file paths, duration, and user-provided notes for prosody analysis.

- **OnboardingEntity:** Stores baseline demographic data and initial psychological assessments.

3.4.3 Schema Evolution and Migrations

To ensure system stability as new features are added, the database employs a rigorous Migration Protocol. The system is currently at Version 8.

3.4.4 Data Lifecycle and Sovereignty

The persistence layer is designed with a Privacy-by-Design approach:

1. Ingestion: Raw telemetry from the Native Android Layer is written to specialized DAO (Data Access Object) interfaces.
2. Transformation: Background tasks trigger the derivation of biomarkers (e.g., converting raw location logs into a daily "Routine Index").
3. The Right to be Forgotten: The EncryptedPersistenceManager includes a "Data Purge" trigger. When activated, it clears the AppDatabase and deletes the encryption keys from the Keystore, rendering all previously stored data permanently irrecoverable.

3.5 Access Control and Security

ScubaMind implements a "Defense-in-Depth" security architecture. Given the highly sensitive nature of digital phenotyping data, the system ensures that information is protected at rest and during processing, with strict access controls governing user privacy.

3.5.1 Multi-Layered Data Security

The security framework is divided into two primary layers of protection on the device:

- **At-Rest Encryption (Hardware-Backed):** As detailed in the persistence layer, the database utilizes AES-256 encryption via SQLCipher. The unique passphrase for this encryption is stored in EncryptedSharedPreferences, which is anchored to the Android Keystore System. This ensures that the cryptographic keys are stored in a Trusted Execution Environment (TEE) or a dedicated StrongBox hardware security module, making them inaccessible even to a rooted operating system or external forensic tools.
- **Sandbox Isolation:** ScubaMind operates within the Android Application Sandbox. This ensures that no other third-party application on the device can access ScubaMind's internal storage, its specialized keyboard (IME) data, or its private voice recording files. Each component is isolated, and data exchange between the Native and Flutter layers is handled through secure, internal communication channels.

3.5.2 Authentication and Authorization

Access to the application's interface and its insights is restricted to the legitimate user through the following mechanisms:

ScubaMind adheres to the principle of least privilege. Users must explicitly authorize individual data streams (e.g., GPS, Microphone, Usage Stats, Step Counts). These permissions can be revoked at any time through the Privacy & Permission Hub, which immediately triggers the Background Service Orchestrator to cease data collection for that specific sensor.

3.5.3 Data Privacy and Sovereignty

In alignment with privacy-by-design principles, ScubaMind provides the following sovereignty features:

- **Local Processing Priority:** The AI Module performs all heavy analysis (prosody, linguistic, and phenotyping) on-device. Raw voice recordings and raw keyboard text never leave the device hardware. Only anonymized, high-level context (e.g., "Activity levels are 30% lower than the weekly average") is shared with the LLM API to facilitate supportive dialogue.
- **Cryptographic Data Purge:** The user has a "master switch" to delete all local data. This operation performs a secure wipe of the AppDatabase and resets the EncryptedSharedPreferences. By destroying the encryption keys, the system ensures that the data is rendered cryptographically unrecoverable, fulfilling the "Right to be Forgotten."

4. Subsystem Services

4.1 Mobile App Subsystem

The mobile app subsystem constitutes the main user-facing part of ScubaMind and is implemented as a Flutter-based mobile application. This subsystem is responsible for presenting behavioral insights, collecting user inputs, managing privacy preferences, and providing access to supportive wellbeing features through a unified interface. Similar to the role of the client subsystem in the example report, the mobile app acts as the interaction layer between the user and the underlying native sensing, storage, and analytics components of the system.

The subsystem consists of several primary screens and interface groups. The splash and onboarding screens guide the user through the initial setup process, including baseline information collection and system preparation. After onboarding, the user accesses the main application shell, which includes the Home, Goals, Chat, and Profile sections. The Home screen serves as the central dashboard and presents high-level outputs such as risk-related summaries, activity information, sleep indicators, weekly insights, and personalized recommendations. The Goals screen allows the user to view, create, edit, and complete wellbeing goals, while also presenting suggested goals and badge-oriented motivational

elements. The Chat screen provides a conversational assistant interface for supportive dialogue, and the Profile section gives access to settings and privacy controls. In addition, the privacy-related pages allow the user to configure which behavioral signals may be collected and which applications may be analyzed by the keyboard subsystem.

User interaction in the mobile app subsystem is centered around simple and continuous engagement. The user can navigate among the main sections through the bottom navigation structure, review daily and weekly summaries, inspect sleep and activity details, manage goals, and interact with the assistant through text-based conversation. The user may also control the sensing behavior of the system by enabling or disabling specific data sources such as steps, sleep, phone usage, location, message analysis, and journaling. Through the privacy interface, the user is given direct control over data sovereignty, which is consistent with the local-first and privacy-preserving design principles defined in earlier sections of the report.

In terms of data transmission, the mobile app subsystem primarily exchanges information with the native Android layer and local persistence components. User-generated data such as onboarding responses, privacy settings, goal definitions, manual interactions, and chat messages are forwarded to local repositories or native platform channels for storage and further processing. Conversely, processed or aggregated data such as daily step counts, sleep summaries, service states, and risk-related outputs are retrieved from native channels and presented in the interface. This structure allows the mobile app to remain lightweight and responsive while delegating sensor access, background data collection, and low-level storage operations to the native subsystem.

Overall, the mobile app subsystem provides the main access point to the ScubaMind platform. It combines dashboard visualization, privacy control, goal management, and supportive interaction into a single cohesive interface, ensuring that the user can both benefit from the system's analytical capabilities and remain in control of how personal data is used.

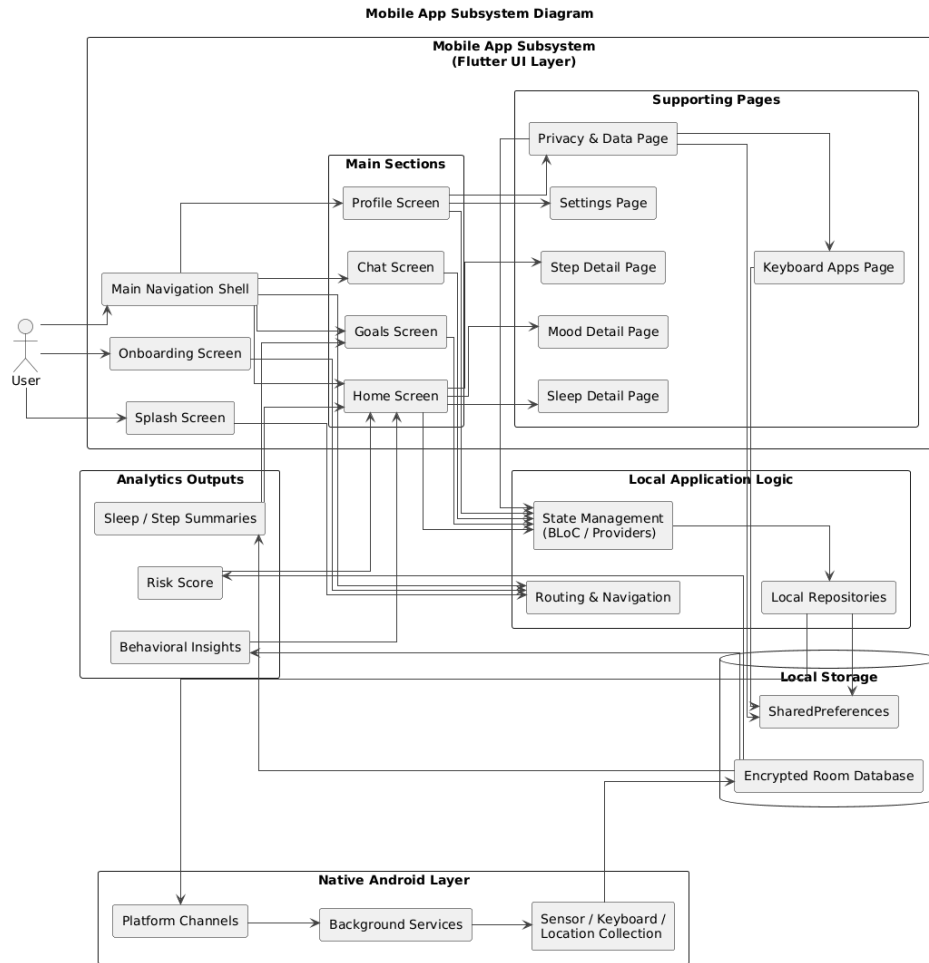


Figure 2. Mobile App Subsystem Diagram

4.2 Backend Subsystem

The backend subsystem of ScubaMind is designed differently from a conventional cloud-based backend architecture. Since the system follows a local-first design, the backend responsibilities are handled primarily by the native Android layer running on the user's device. In this sense, the backend subsystem acts as the internal service and processing coordinator of the application. It receives requests from the mobile app subsystem, manages access to device-level resources, controls background services, and coordinates the storage and retrieval of behavioral data required by the analytics components.

The backend subsystem is responsible for several core functions. First, it provides the communication bridge between the Flutter-based mobile interface and the native Android services through platform channels. These internal interfaces play a role similar to API endpoints in a traditional distributed system, since they expose specific operations such as retrieving step counts, reading sleep summaries, accessing onboarding information, managing voice recordings, obtaining installed applications for keyboard configuration, and starting or stopping location-related services. In this way, the backend subsystem acts as the execution layer that responds to requests from the user-facing mobile application and returns the requested data in a structured form.

A second responsibility of the backend subsystem is service orchestration and processing logic. The subsystem manages foreground services for step tracking, sleep monitoring, location collection, and keyboard-based text collection. It determines whether these services should be active according to privacy settings and stored user preferences. It also handles boot-time recovery behavior so that previously enabled services can continue operating after a device restart. Beyond simply collecting data, the backend subsystem performs intermediate processing tasks such as updating daily step and sleep values, buffering keyboard text, computing mobility-related statistics, and organizing daily multimodal records before these are consumed by the analytics layer. Thus, the backend does not only transmit data, but also performs the operational logic required to keep the sensing pipeline consistent and efficient.

The backend subsystem additionally manages local data persistence and retrieval. It communicates with the encrypted Room database and preference-based configuration storage to ensure that raw and processed data remain accessible to the rest of the system while still being protected. Inbound data includes user settings, onboarding answers, service control signals, and raw sensor outputs. Outbound data includes aggregated daily values, historical summaries, and stored behavioral records requested by the mobile app or analytics modules. This makes the backend subsystem the central coordination point between user interaction, passive sensing, and persistent storage.

Although the current architecture prioritizes on-device backend functionality, the subsystem is also designed to remain extensible for future remote integrations. In particular, the optional LLM companion described in the previous architecture section may require communication with an external service for supportive conversational responses. Even in such a case, the backend subsystem would remain responsible for preparing the required context, limiting transmitted information to high-level and privacy-preserving summaries, and ensuring that the main sensing and analysis pipeline can continue functioning independently of cloud connectivity.

Overall, the backend subsystem serves as the operational backbone of ScubaMind. It replaces the role of a traditional server backend with an on-device native coordination layer that manages service execution, internal interfaces, local processing logic, and secure persistence while supporting the privacy-preserving goals of the system.

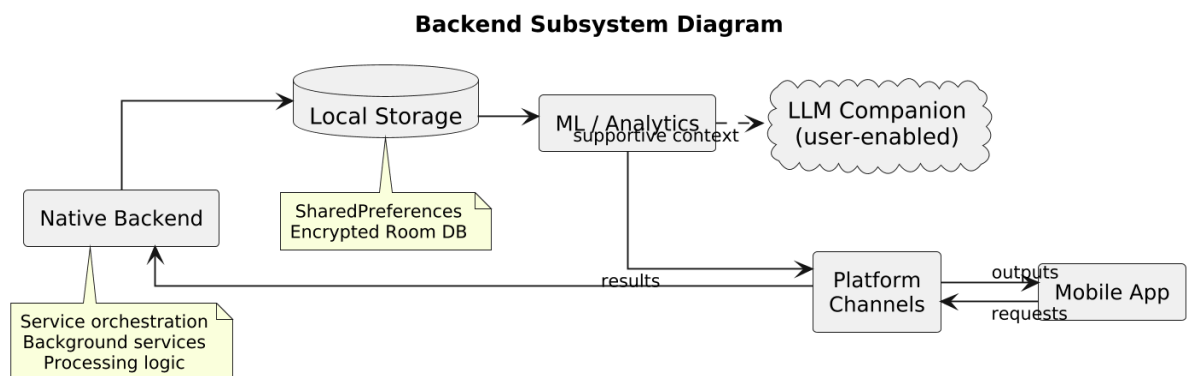


Figure 3. Backend Subsystem Diagram

4.3 ML / Analytics Subsystem

The ML / Analytics subsystem of ScubaMind is responsible for transforming raw behavioral and physiological data into interpretable mental health insights. In the overall architecture of the system, this subsystem serves as the analytical core that processes multimodal inputs collected through the native Android layer and produces outputs that can be presented to the user through the mobile application. Its main role is not to provide clinical diagnosis, but rather to generate awareness-oriented indicators such as behavioral summaries, mood-related interpretations, and personalized risk-related outputs.

The subsystem operates on data gathered from multiple sources within the system. These sources include step and activity information, sleep-related measurements, mobility patterns derived from location samples, onboarding-based baseline information, keyboard-derived text signals, and other behavioral records stored locally on the device. By combining data from different sensing modalities, the analytics subsystem supports the digital phenotyping approach described in the earlier architectural sections. This allows the system to move beyond single-signal monitoring and instead evaluate broader behavioral trends that may be associated with changes in mental well-being.

One of the primary responsibilities of this subsystem is data processing and feature preparation. Before meaningful outputs can be generated, raw records must be organized, aggregated, and converted into analytical features. Examples of such transformations include computing daily activity summaries, deriving sleep duration values, extracting mobility-related indicators such as distance or routine-related measures, and consolidating multimodal daily records into a form suitable for higher-level interpretation. In this sense, the subsystem acts as the stage where raw sensor logs become structured behavioral representations.

A second key responsibility is model inference and insight generation. Based on the prepared features, the subsystem produces outputs such as behavioral summaries, mood-related categories, and risk scores that reflect observed trends in the user's recent behavior. These outputs are intended to be understandable, privacy-preserving, and supportive rather than diagnostic. The subsystem is therefore designed to emphasize on-device inference and local processing, ensuring that sensitive raw inputs such as text, location traces, and daily behavioral logs remain within the user's device boundary as much as possible.

The ML / Analytics subsystem also provides the bridge between passive sensing and user-facing feedback. Once analytical results are generated, they are made available to the mobile app subsystem in the form of dashboard indicators, summary values, and recommendation-oriented outputs. These results may also be used as contextual support for the LLM Companion, enabling more personalized and reflective interactions without requiring the transmission of raw behavioral data. In this way, the analytics subsystem supports both direct visualization and higher-level supportive interaction within the application.

Overall, the ML / Analytics subsystem forms the intelligence layer of ScubaMind. It receives multimodal behavioral data, processes and interprets that data through feature extraction

and analytical inference, and produces meaningful outputs that help the user better understand changes in daily habits and wellbeing-related patterns. This makes it a central subsystem in fulfilling the project's goal of privacy-preserving, continuous mental health monitoring.

ML / Analytics Subsystem Diagram

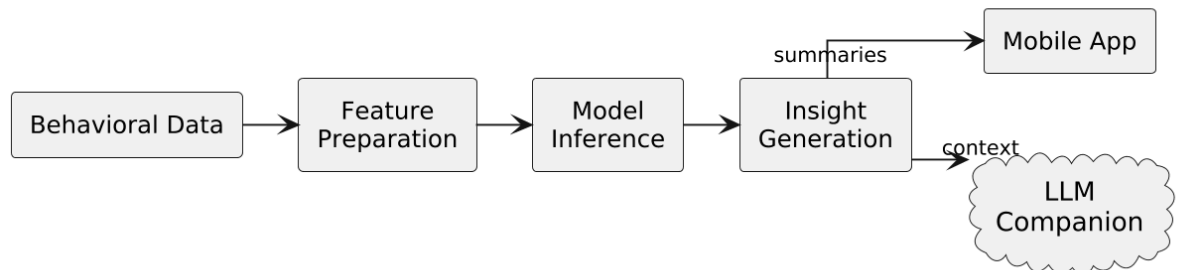


Figure 4. ML/Analytics Subsystem Diagram

4.4 Method Level Design

4.4.1 Core Algorithms

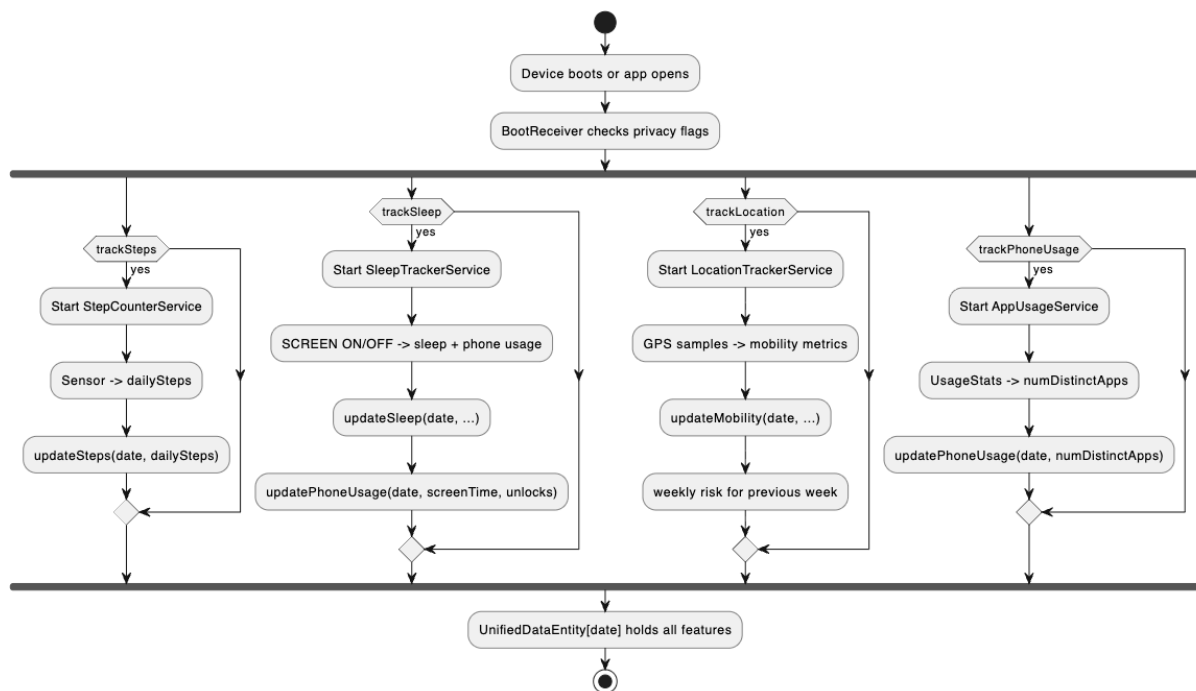


Figure 5. Unified Daily Data Update Diagram

This diagram illustrates the unified daily data update mechanism of the ScubaMind system. When the device boots or the application starts, the BootReceiver checks the user's privacy settings to determine which sensing modules are allowed to run. Based on these permissions, different background services such as StepCounterService, SleepTrackerService, LocationTrackerService, and AppUsageService are activated. Each service collects a specific type of behavioral data (e.g., steps, sleep patterns, mobility metrics, and phone usage statistics) and updates the corresponding fields in the UnifiedDataEntity through repository update functions. All collected modalities are aggregated into a single daily record, allowing the system to maintain a unified representation of the user's behavioral features for further analysis and model processing.

4.4.2 Sequence Diagrams

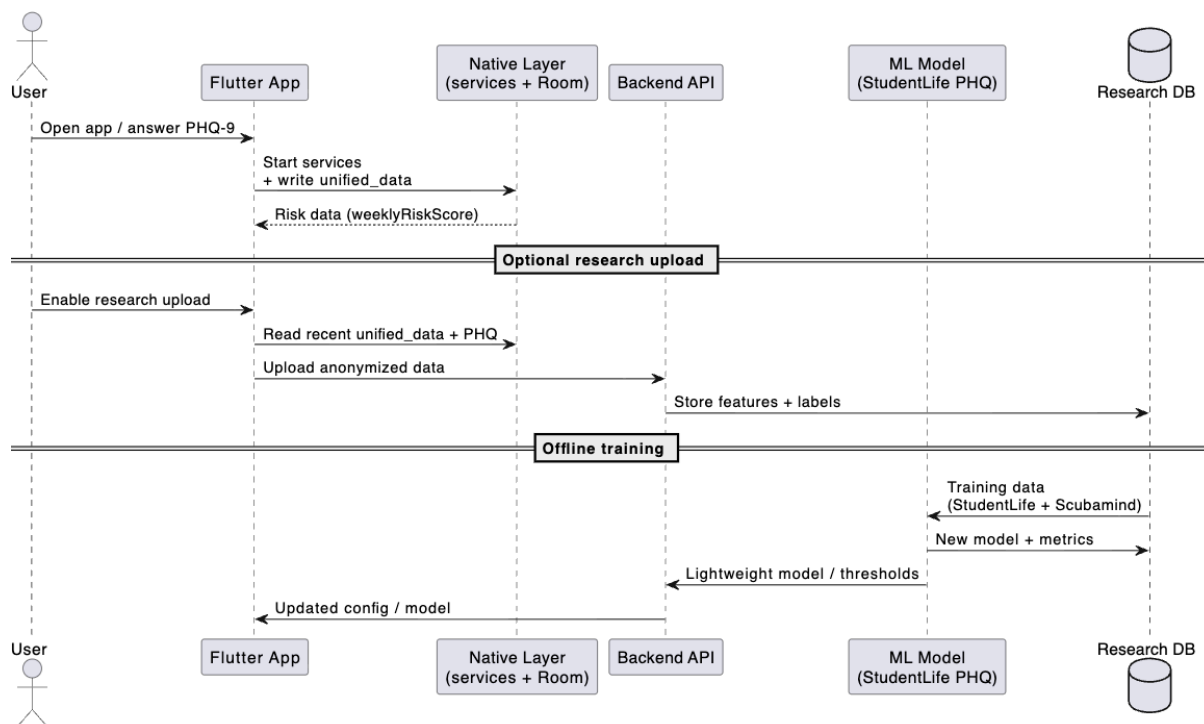


Figure 6. Sequence Diagram of the ScubaMind system

This sequence diagram illustrates the interaction between the main components of the ScubaMind system. When the user opens the application or completes the PHQ-9 questionnaire, the Flutter app triggers the native layer to start background services and store collected behavioral data in the local unified_data database. The system computes risk-related indicators such as the weekly risk score and makes them available to the application. If the user enables the optional research participation feature, anonymized data and PHQ labels are uploaded to the backend, where they are stored in the research database. Offline training is then performed using aggregated datasets, and the resulting lightweight model parameters or thresholds are sent back to the application configuration for future analyses.

4.4.3 Activity Diagrams

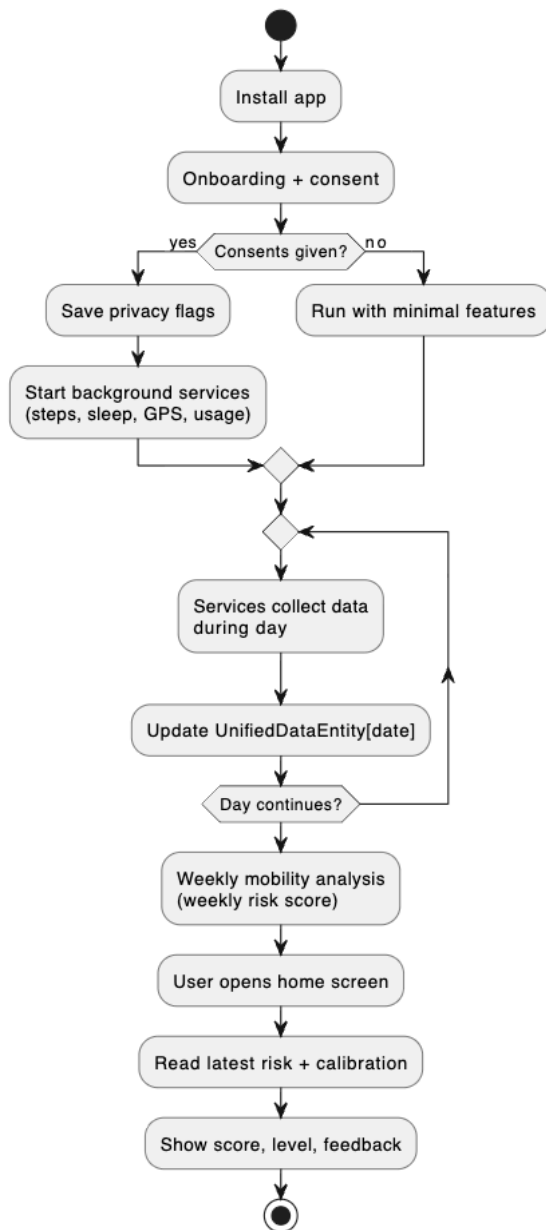


Figure 7. Activity Diagram – User Flow of the ScubaMind system

This activity diagram illustrates the overall user interaction and system workflow of the ScubaMind application. After installing the app, the user completes the onboarding process and provides consent for data collection. If consent is granted, the system activates background services that collect behavioral data such as steps, sleep, mobility, and phone usage throughout the day. These signals are continuously aggregated and stored in the UnifiedDataEntity for the corresponding date. Periodically, mobility-based analysis is performed to compute the weekly risk score. When the user opens the home screen, the system retrieves the latest risk information and calibration parameters, and displays the corresponding score, risk level, and feedback to the user.

4.4.4 Class Diagram

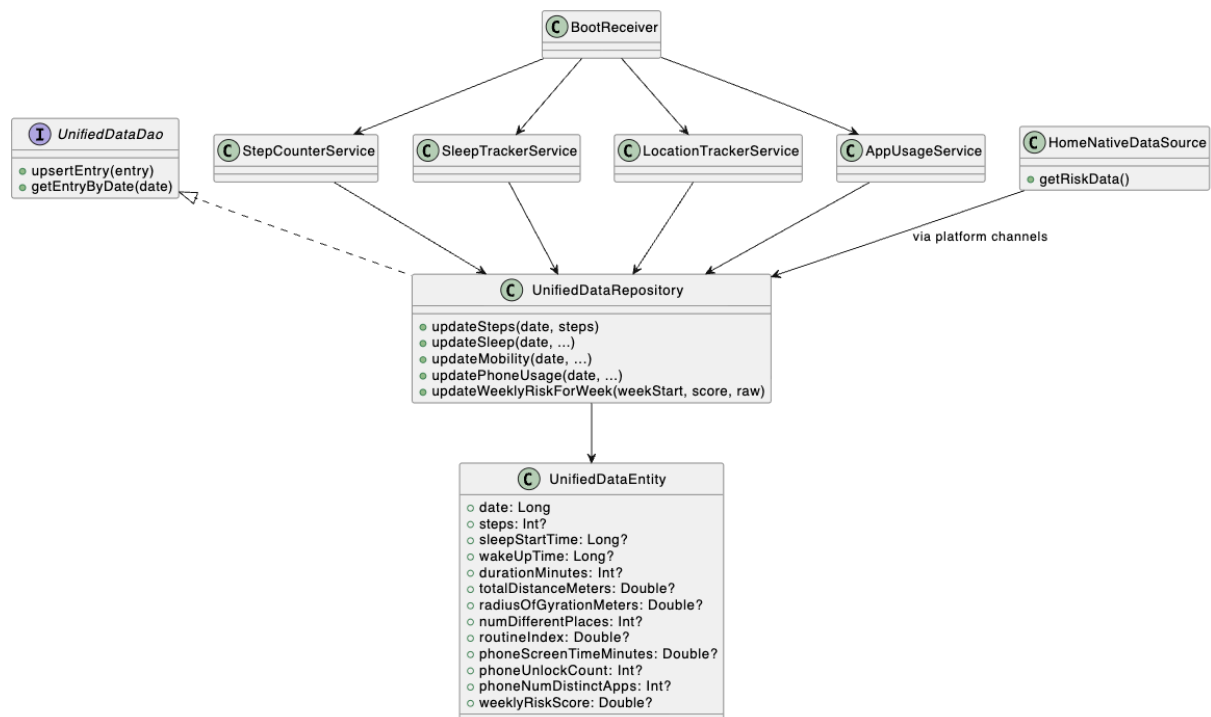


Figure 8. Class Diagram – Core data & services of the ScubaMind system

This class diagram shows the core Android data model and services of ScubaMind. UnifiedDataEntity stores one row of daily, unified features (steps, sleep, mobility, phone usage, and weekly risk score). UnifiedDataDao is the Room DAO, and UnifiedDataRepository is the main abstraction used by background services. The four foreground services (StepCounterService, SleepTrackerService, LocationTrackerService, and AppUsageService) all write into the repository, while BootReceiver starts the services after boot based on user privacy settings. On the Flutter side, HomeNativeDataSource reads the aggregated risk information through platform channels that query the same repository.

4.4.5 Use Case Diagram

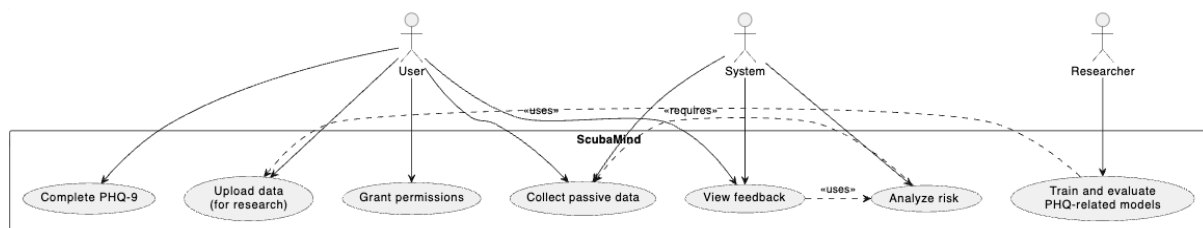


Figure 9. Use Case Diagram – Main actors and use cases of the ScubaMind system

This use case diagram summarizes the high-level functionality and stakeholders of the system. The User grants permissions, allows passive data collection, completes the PHQ-9 questionnaire, views risk feedback, and optionally uploads anonymised data for research. The System (mobile app and background services) performs passive data collection and risk

analysis, and delivers feedback back to the user. The Researcher uses the uploaded data to train and evaluate PHQ-related models. “Analyze risk” depends on “Collect passive data”, and “View feedback” uses the output of the risk analysis.

4.4.6 Data Flow Diagrams

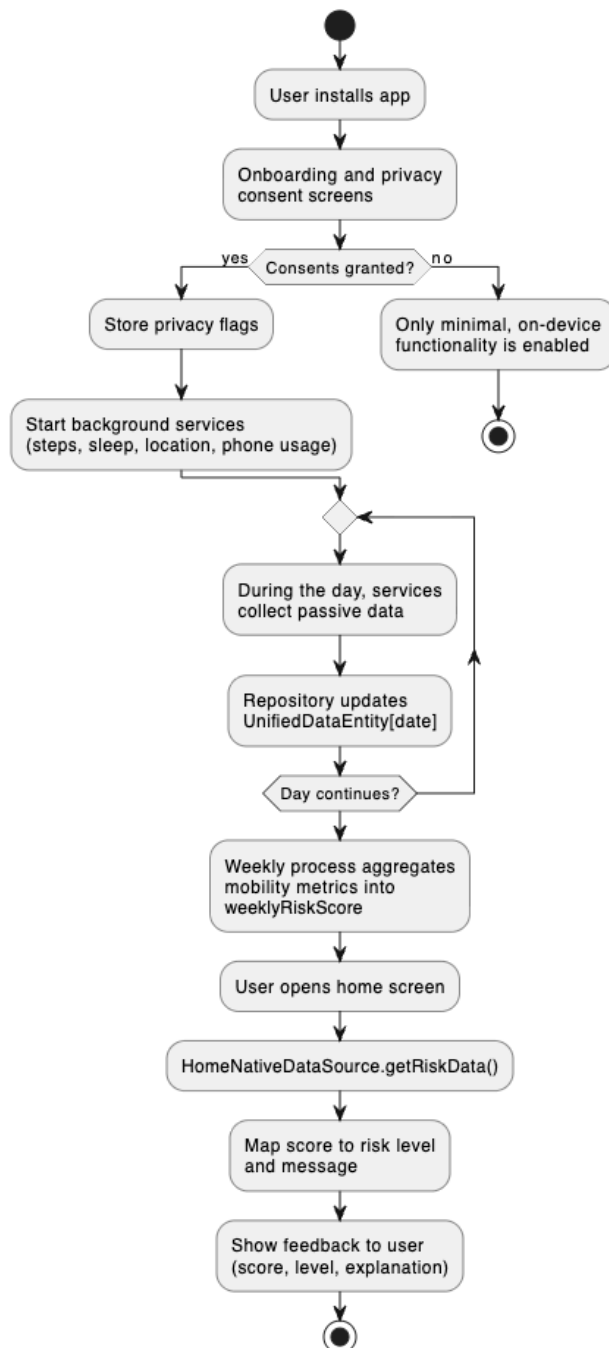


Figure 10. Activity / Use-Case Flow – From consents to feedback of the ScubaMind system

This activity diagram describes the end-to-end user flow from onboarding to feedback. After installation, the user goes through consent screens; if sufficient consents are granted, the Android services start and continuously collect passive data (steps, sleep, GPS, and phone

usage) and update UnifiedDataEntity. Periodically, the system aggregates these signals into a weekly mobility-based risk score. When the user opens the home screen, the app retrieves the latest risk data, maps the score to a qualitative risk level, and presents an interpretable explanation back to the user.

4.5. Development Methodology

The development of the ScubaMind system follows a structured and sequential software development approach. Considering the sensitive nature of the application, the complexity of integrating mobile sensing, artificial intelligence, and privacy-preserving mechanisms, a well-defined and predictable development methodology was necessary.

For this reason, the Waterfall development model was adopted as the primary methodology for the project. The Waterfall model allows the project team to progress through clearly defined phases in a linear and controlled manner. Each phase produces specific deliverables and documentation that serve as inputs for the next phase.

Since the project is carried out within the scope of a Senior Design Project, documentation and verification of each phase are important milestones. Therefore, the Waterfall methodology provides a suitable structure to ensure traceability between requirements, system design, implementation, and testing stages.

In the context of the ScubaMind system, the development process was divided into the following major phases: Analysis, Design, Implementation, Testing, and Deployment. Each phase contributes to the gradual transformation of system requirements into a fully functioning mobile application.

4.5.1 Development Model

The ScubaMind project adopts the Waterfall Development Model, which is a traditional and structured software development methodology where development progresses sequentially through predefined phases.

In this model, each stage must be completed before the next stage begins. The outputs produced in one phase become the inputs of the following phase. This structured workflow helps ensure that system requirements are clearly defined before design decisions are made, and that the implementation process remains aligned with the original system objectives.

The choice of the Waterfall model was influenced by several factors specific to the project:

- The project requires extensive documentation, including Analysis Report, Detailed Design Report, and Final Report.
 - System requirements must be clearly specified before implementation, particularly due to the sensitive nature of mental health data.
 - The architecture involves multiple subsystems such as mobile sensing, AI analysis modules, data storage, and UI components, which require careful design before

development begins.

- The academic structure of the project naturally follows a phase-based evaluation process.

Using the Waterfall model allowed the team to establish a clear roadmap from requirements to implementation while maintaining strong documentation and traceability between phases.

4.5.2 Development Phases

The development of the ScubaMind system was carried out through a structured process consisting of five major phases: Analysis, Design, Implementation, Testing, and Deployment. Each phase gradually transformed the initial problem definition into a functional mobile system capable of collecting behavioral signals, analyzing them using AI models, and providing user feedback while preserving user privacy.

Analysis Phase

The analysis phase focused on understanding the challenges associated with monitoring mental well-being through digital technologies. Existing mental health monitoring approaches were examined, including traditional clinical questionnaires and recent digital phenotyping systems.

Based on this analysis, the main objective of the ScubaMind system was defined as developing a privacy-preserving mobile platform that can infer behavioral patterns from passive sensing signals and provide mental well-being insights to users.

During this phase, system requirements were identified and documented. These included both functional requirements, such as sensor data collection, behavioral analysis, and recommendation generation, and non-functional requirements, including privacy preservation, local data processing, low battery consumption, and usability.

Additionally, the overall system scope, user scenarios, and interaction flows were defined. These outputs formed the foundation for the architectural design of the system.

Design Phase

In the design phase, the requirements defined during the analysis stage were translated into a detailed system architecture. The system was decomposed into several subsystems in order to separate responsibilities and improve modularity.

The architecture of ScubaMind consists of the following main components:

- Mobile User Interface Layer (Flutter) responsible for user interaction and visualization of insights
 - Native Android Data Collection Layer (Kotlin) responsible for collecting passive sensing data such as location, mobility metrics, and device activity
 - Local Data Storage Layer responsible for securely storing collected data on the device
 - AI Analysis Module responsible for processing behavioral signals and generating risk indicators

- LLM Companion Module responsible for generating contextual recommendations and supportive messages

Special attention was given to designing a local-first architecture, where sensitive user data is processed primarily on the device rather than being transmitted to external servers. This architectural decision significantly improves privacy protection and reduces potential risks associated with cloud-based data storage.

During the design phase, system structure, component interactions, and data flows were defined through architectural models and diagrams. These design artifacts served as the blueprint for the implementation stage.

Implementation Phase

The implementation phase focused on developing the functional components of the ScubaMind system according to the architecture defined in the design phase.

The mobile application interface was implemented using the Flutter framework, enabling a responsive and modern user interface. The native Android layer was developed using Kotlin, which allowed direct access to device sensors and system-level services required for passive sensing.

Background services were implemented to collect behavioral signals such as mobility patterns and device activity while minimizing battery consumption. The collected data is stored locally and then processed by the AI analysis module, which computes behavioral indicators and risk scores.

Integration between the Flutter UI layer and the native Android layer was achieved through Flutter Method Channels, allowing efficient communication between the two environments.

At the current stage of the project, a working prototype of the ScubaMind system has been implemented, demonstrating the core functionality of data collection, behavioral analysis, and user feedback generation.

Testing Phase

The testing phase focused on validating the functionality and integration of different system components. Since the project is currently at the prototype stage, testing was primarily conducted through functional verification and scenario-based evaluation.

Several aspects of the system were tested, including:

- Correct operation of sensor data collection mechanisms
- Reliable storage of collected behavioral data
- Proper functioning of AI-based analysis modules
- Correct communication between Flutter and the native Android layer
- User interface usability and interaction flow

Testing was performed through controlled usage scenarios to ensure that each subsystem operates as expected and that the overall system behaves consistently.

Due to the prototype nature of the system, testing focused on validating system functionality rather than implementing a fully automated test suite.

Deployment Phase

The deployment phase focuses on preparing the ScubaMind system for demonstration and evaluation within the scope of the senior design project.

The current version of the system represents a functional prototype that integrates the main architectural components, including passive sensing, behavioral analysis, and user interaction modules. The application is deployed on mobile devices for demonstration and testing purposes.

Future deployment stages may involve broader user studies, system optimization, and integration with additional data sources. However, within the scope of this project, the deployment phase primarily involves delivering a working prototype and presenting the system architecture and capabilities.

5. Test Cases

5.1 Functional Test Cases

5.1.1 TC-F01 — Home page loads with mental health overview

Test ID	TC-F01	Category	Functional	Severity	Critical
Objective	Verify the Home page loads and displays the mental health overview, risk level, and mini stat cards (Mood, Activity, Sleep).				
Steps	1. Launch the app and complete onboarding/splash. 2. Observe the Home page. 3. Wait for data to load.				
Expected	1. Page loads without errors. 2. AppBar shows ScubaMind. 3. Risk level and Risk score are visible. 4. Three mini stat cards are rendered. 5. Weekly insights and AI recommendations sections are visible.				

Table 2 - TC-F01 Test Case Table

5.1.2 TC-F02 — Pull-to-refresh reloads all Home blocs Test

Test ID	TC-F02	Category	Functional	Severity	Major
Objective	Verify pull-to-refresh triggers a refresh of HomeBloc, StepBloc, and SleepBloc.				
Steps	1. Open Home page with loaded data. 2. Pull down (scroll overscroll). 3. Release and wait.				
Expected	1. LoadHomeDataEvent, StepStarted, SleepStarted are dispatched. 2. Mini-cards briefly show ... then reload. 3. Page does not crash.				

Table 3 - TC-F02 Test Case Table

5.1.3 TC-F03 — Voice recordings list loads from DB

Test ID	TC-F03	Category	Functional	Severity	Critical
Objective	Verify VoiceRecordingsListPage loads and lists saved recordings from the local Room database.				
Steps	1. Ensure at least one recording exists in DB. 2. Navigate to Voice Recordings. 3. Observe the list.				
Expected	1. LoadVoiceRecordingsEvent dispatched on init. 2. CircularProgressIndicator shown while loading. 3. Cards appear with title, date, duration, file size.				

Table 4 - TC-F03 Test Case Table

5.1.4 TC-F04 — Play/Pause toggles correctly

Test ID	TC-F04	Category	Functional	Severity	Critical
Objective	Verify tapping the play button plays audio and a second tap pauses it.				
Steps	1. Open Voice Recordings with at least one recording. 2. Tap play on recording A. 3. Observe the button icon. 4. Tap it again.				
Expected	1. Button icon changes to pause; audio plays. 2. Audio pauses; icon returns to play.				

Table 5 - TC-F04 Test Case Table

5.1.5 TC-F05 — Confirm delete removes recording

Test ID	TC-F05	Category	Functional	Severity	Critical
Objective	Verify confirming deletion dispatches DeleteVoiceRecordingEvent and removes the recording from the list.				
Steps	1. Open Voice Recordings with at least one recording. 2. Tap delete icon on recording A. 3. Tap Delete in the confirmation dialog.				
Expected	1. DeleteVoiceRecordingEvent(id) dispatched. 2. Recording card disappears from the list. 3. If last recording, empty state shown.				

Table 6 - TC-F05 Test Case Table

5.1.7 TC-F06 —SpeechToTextModel loads encoder, decoder, and tokenizer

Test ID	TC-F06	Category	Functional	Severity	Critical
Objective	Verify SpeechToTextModel.load() initializes ONNX encoder/decoder sessions and the WhisperTokenizer.				
Steps	1. Instantiate SpeechToTextModel with a valid OnnxSessionManager. 2. Call model.load(context). 3. Check model.isLoaded().				
Expected	1. encoder_model.onnx, decoder_model_merged.onnx, vocab.json load without exceptions. 2. isLoaded() returns true. 3. Log contains Whisper tiny loaded successfully.				

Table 7 - TC-F06 Test Case Table

5.1.7 TC-F07 —Inference returns empty string for silent audio

Test ID	TC-F07	Category	Functional	Severity	Major
Objective	Verify infer() returns empty transcription when audio RMS is below the silence threshold (0.001f).				
Steps	1. Load the model. 2. Create AudioSegment with near-zero PCM samples. 3. Call model.infer(ModelInput(audioSegment)).				
Expected	1. Output is ModelOutput.Success. 2. TranscriptSegment.text is empty string. 3. Log shows Audio segment too silent.				

Table 8 - TC-F07 Test Case Table

5.1.8 TC-F08 — Chat page sends and displays messages

Test ID	TC-F08	Category	Functional	Severity	Critical
Objective	Verify typing a message and tapping Send adds it to the conversation and the chat scrolls to the new message.				
Steps	1. Navigate to the Chat screen. 2. Tap text input. 3. Type Hello, how are you? 4. Tap Send.				
Expected	1. Message appears with user-bubble style. 2. Input field clears after sending. 3. Chat scrolls to latest message.				

Table 9 - TC-F08 Test Case Table

5.1.9 TC-F09 — PHQ questionnaire loads and submits

Test ID	TC-F09	Category	Functional	Severity	Critical
Objective	Verify the PHQ screen renders all questionnaire items and submitting persists the results.				
Steps	1. Navigate to the PHQ screen. 2. Select an answer for each question. 3. Tap Submit.				
Expected	1. All PHQ items are listed with selectable options. 2. No validation errors on submit. 3. Results stored in local DB and user navigated to results screen.				

Table 10 - TC-F09 Test Case Table

5.1.10 TC-F10 — Onboarding flow completes and is not shown again

Test ID	TC-F10	Category	Functional	Severity	Critical
Objective	Verify the onboarding flow presents all steps, completes successfully, and is not shown again on next launch.				
Steps	1. Launch app as new user (clear app data). 2. Navigate through each step by tapping Next. 3. Close and relaunch the app.				
Expected	1. Each onboarding screen loads without error. 2. After last step, user is directed to Home. 3. On relaunch, splash navigates directly to Home, not onboarding.				

Table 11 - TC-F10 Test Case Table

5.1.11 TC-F11 —Interactive elements have adequate touch target

Test ID	TC-F11	Category	Functional / Component	Severity	Major
Objective	Verify a recording with non-null transcription shows the transcription section on its card.				
Steps	1. Open Voice Recordings where at least one recording has a transcription in DB. 2. Observe the card.				
Expected	1. Transcription label with icon shown. 2. Text displayed (max 5 lines, ellipsis on overflow). 3. No spinner shown for that card.				

Table 12 - TC-F11 Test Case Table

5.1.12 TF-F12 —Transcribing shown while transcription is null

Test ID	TC-F12	Category	Functional / Component	Severity	Major
Objective	Verify a recording with transcription == null shows a Transcribing spinner.				
Steps	1. Open Voice Recordings where a recording has transcription = null. 2. Observe the card.				
Expected	1. Small CircularProgressIndicator with Transcribing italic text shown. 2. No transcription text shown.				

Table 13 - TC-F12 Test Case Table

5.1.13 TC-F13 —Inference fails gracefully when model not loaded

Test ID	TC-F13	Category	Functional / ML Model	Severity	Critical
Objective	Verify calling infer() before load() returns ModelOutput.Error rather than an exception.				
Steps	1. Instantiate SpeechToTextModel without calling load(). 2. Call model.infer() with valid audio.				
Expected	1. isLoaded() returns false. 2. Output is ModelOutput.Error. 3. Error message contains Models not loaded. 4. No crash.				

Table 14 - TC-F13 Test Case Table

5.1.14 TC-F14 —Chat page loads and displays message list

Test ID	TC-F14	Category	Functional / Component	Severity	Critical
Objective	Verify the Chat page loads and shows conversation history.				
Steps	1. Navigate to the Chat screen. 2. Wait for data to load.				
Expected	1. Page loads without error. 2. Existing messages displayed in chronological order. 3. Input field is visible and focusable.				

Table 15 - TC-F14 Test Case Table

5.1.15 TC-F15 —Empty message is not sent

Test ID	TC-F15	Category	Functional / Component	Severity	Minor
Objective	Verify tapping Send with an empty input does nothing.				
Steps	1. Open Chat. 2. Leave input empty. 3. Tap Send.				
Expected	1. No message added to the chat. 2. No error shown. 3. Input field remains empty.				

Table 16 - TC-F15 Test Case Table

5.2 Non-Functional Test Cases

5.2.1 TC-P01 — ASR inference completes within 10 seconds

Test ID	TC-P01	Category	Performance	Severity	Critical
Objective	Verify that end-to-end Whisper ASR inference for a 30-second audio clip completes within 10 seconds on a mid-range Android device.				
Steps	1. Load SpeechToTextModel on a target device (mid-range, e.g. Pixel 5). 2. Prepare a 30-second PCM audio segment with clear speech. 3. Record System.currentTimeMillis() before and after model.infer(). 4. Calculate elapsed time.				
Expected	1. Total inference time (encoder + decoder + tokenizer) <= 10,000 ms. 2. Output is ModelOutput.Success with non-empty transcription. 3. No OOM or timeout exception.				

Table 17 - TC-P01 Test Case Table

5.2.2 TC-P02 — Voice recordings list loads within 2 seconds for 50+ entries

Test ID	TC-P02	Category	Performance	Severity	Major
Objective	Verify VoiceRecordingsListPage renders all recording cards within 2 seconds when 50+ recordings exist in the database.				
Steps	1. Insert 50 voice recording rows into the local Room DB. 2. Navigate to Voice Recordings screen. 3. Measure time from navigation to full list render.				
Expected	1. Full list rendered and visible within 2,000 ms. 2. Scrolling is smooth (no frame drops > 16 ms). 3. No ANR dialog appears.				

Table 18 - TC-P02 Test Case Table

5.2.3 TC-P03 — Home page initial load completes within 3 seconds

Test ID	TC-P03	Category	Performance	Severity	Major
Objective	Verify that the Home page fully loads and displays all data within 3 seconds of navigation.				
Steps	1. Launch the app and navigate to Home page. 2. Measure time from page push to all three mini-cards showing data. 3. Repeat 5 times and record average.				
Expected	1. Average load time <= 3,000 ms. 2. Risk level, step count, and sleep hours display correctly. 3. No timeout errors in logs.				

Table 19 - TC-P03 Test Case Table

5.2.4 TC-S01 — Voice recordings stored in private app directory

Test ID	TC-S01	Category	Security	Severity	Critical
Objective	Verify that voice recording audio files are saved inside the app private directory so other apps cannot read them.				
Steps	1. Record and save a voice memo. 2. Use adb shell to inspect the file path stored in the DB. 3. Attempt to read the file from a second unprivileged app.				
Expected	1. File path is inside /data/data/com.scuba.scuba/ or app-specific external storage. 2. File is not accessible by unprivileged apps without root. 3. No world-readable permission on the file.				

Table 20 - TC-S01 Test Case Table

5.2.5 TC-S02 — Unauthorized DB access attempt is blocked

Test ID	TC-S02	Category	Security	Severity	Critical
Objective	Verify that a third-party app without root cannot directly read the Scubamind Room database file.				
Steps	1. Install a file-browser app without root privileges. 2. Attempt to navigate to /data/data/com.scuba.scuba/databases/. 3. Observe whether files are readable.				
Expected	1. File browser cannot list or open the database files. 2. Permission denied error is shown. 3. No sensitive data is leaked.				

Table 21 - TC-S02 Test Case Table

5.2.6 TC-S03 — PHQ data not stored in plain-text shared preferences

Test ID	TC-S03	Category	Security	Severity	Critical
Objective	Verify that sensitive PHQ responses and risk scores are not written to SharedPreferences or any unencrypted accessible storage.				
Steps	1. Complete a PHQ questionnaire. 2. Inspect SharedPreferences XML files via adb pull. 3. Inspect Room database file for PHQ fields.				
Expected	1. PHQ answers stored only in Room database inside the app sandbox. 2. No PHQ data in SharedPreferences. 3. Database file is not world-readable.				

Table 22 - TC-S03 Test Case Table

5.2.7 TC-UX01 — User can start a recording without instructions

Test ID	TC-UX01	Category	Usability	Severity	Major
Objective	Verify that a first-time user can locate and start a voice recording without requiring any instructions or help text.				
Steps	1. Recruit a participant unfamiliar with the app. 2. Ask them to find the voice recording feature and make a recording. 3. Observe navigation path and measure time-to-first-recording.				
Expected	1. User locates the record button within 60 seconds. 2. A recording is successfully saved. 3. No more than 2 wrong navigation attempts before finding the screen.				

Table 23 - TC-UX01 Test Case Table

5.2.8 TC-UX02 — Error messages are clear and actionable

Test ID	TC-UX02	Category	Usability	Severity	Major
Objective	Verify that error states (network failure, DB error, ASR failure) display messages that users can understand and act upon.				
Steps	1. Simulate a DB load failure on the Voice Recordings screen. 2. Observe the error message displayed. 3. Tap the Retry button. 4. Repeat for ASR transcription failure.				
Expected	1. Error message uses plain language (no stack traces or internal codes). 2. A clear action (Retry, Go Back) is offered. 3. After Retry, the app attempts to reload successfully.				

Table 24 - TC-UX02 Test Case Table

5.2.9 TC-UX03 — Interactive elements have adequate touch targets

Test ID	TC-UX03	Category	Usability	Severity	Minor
Objective	Verify that all tappable elements (play, delete, navigation) meet the minimum 48x48 dp touch target size per Android accessibility guidelines.				
Steps	1. Open Voice Recordings screen. 2. Use Android Accessibility Scanner to inspect play, delete, and back buttons. 3. Repeat on Home page for mini-stat cards.				
Expected	1. All touch targets are $\geq 48 \times 48$ dp. 2. Accessibility Scanner reports no touch-target warnings. 3. No overlapping interactive areas found.				

Table 25 - TC-UX03 Test Case Table

5.2.10 TC-R01 — System works under heavy load (concurrent transcriptions)

Test ID	TC-R01	Category	Reliability	Severity	Critical
Objective	Verify that the app remains stable and responsive when 5 voice recordings are queued for transcription simultaneously in the background.				
Steps	1. Record 5 voice clips of 20-30 seconds each in quick succession. 2. Observe the transcription queue via Logcat. 3. Monitor memory usage in Android Studio Profiler. 4. Interact with the UI (navigate between screens) during processing.				
Expected	1. All 5 recordings are eventually transcribed without crash. 2. App UI remains responsive (no ANR). 3. Memory usage does not exceed 500 MB. 4. No OOM exception is thrown.				

Table 26 - TC-R01 Test Case Table

5.2.11 TC-R02 — App recovers from ASR crash without data loss

Test ID	TC-R02	Category	Reliability	Severity	Critical
Objective	Verify that if Whisper inference throws an unexpected exception, the recording audio file and metadata are preserved and the app continues running.				
Steps	1. Mock the decoder session to throw a RuntimeException mid-inference. 2. Trigger transcription of a saved recording. 3. Observe app state after the exception. 4. Verify the recording entry in the DB.				
Expected	1. App catches the exception and logs ModelOutput.Error. 2. App does not crash (no force-close dialog). 3. Voice recording row in DB still exists with original data. 4. transcription field remains null (no corrupted text).				

Table 27 - TC-R02 Test Case Table

5.2.12 TC-R03 — Data persists after unexpected app termination

Test ID	TC-R03	Category	Reliability	Severity	Critical
Objective	Verify that voice recordings and PHQ data saved to the Room database survive an unexpected app kill (process kill via adb).				
Steps	1. Save at least one voice recording and complete one PHQ submission. 2. Force-stop the app via adb shell am force-stop com.scuba.scuba. 3. Relaunch the app. 4. Navigate to Voice Recordings and check PHQ history.				
Expected	1. All previously saved recordings are still listed. 2. PHQ submission data is intact in the DB. 3. No orphaned or corrupted rows.				

Table 28 - TC-R03 Test Case Table

6. Consideration of Various Factors in Engineering Design

The design of ScubaMind was shaped not only by technical requirements but also by broader engineering considerations related to public well-being, safety, ethics, privacy, and sustainability. Since the system operates in the sensitive domain of mental health monitoring, these factors have had a direct impact on both the analysis phase and the detailed design decisions of the project.

ScubaMind is designed as a privacy-preserving, local-first mobile application that collects behavioral and physiological signals from smartphones and wearable devices, analyzes them using on-device artificial intelligence models, and presents the outputs as behavioral insights and risk awareness indicators rather than clinical diagnoses. Because of this design scope, the project is influenced by multiple engineering factors at different levels.

6.1 Constraints

6.1.1 Public Health

Public health is one of the most influential factors in the design of ScubaMind. Depression and related mental health disorders represent a major global health challenge, and the system is intended to support earlier awareness of behavioral changes that may be associated with mental health risks. By enabling continuous and passive monitoring through smartphones and wearable devices, ScubaMind aims to reduce the gap between occasional clinical observations and real-life daily behavioral patterns.

However, the system is not designed as a diagnostic medical tool. For this reason, the outputs of the system are intentionally framed as behavioral risk insights rather than medical

conclusions. This design choice was made to support public health in a responsible way while avoiding unsafe overclaiming.

6.1.2 Public Safety

Public safety is another highly important factor because the system deals with highly sensitive behavioral and mental health-related information. The design must ensure that users are not harmed by inaccurate interpretations, inappropriate recommendations, or privacy violations.

To address this, the system includes several safety-oriented design constraints. First, all outputs are presented in non-diagnostic language. Second, the application must guide users toward professional support instead of acting as a replacement for mental health professionals. Third, if the system detects critical risk indicators, the application should prioritize emergency guidance and supportive resources instead of ordinary interaction flows. In this way, safety considerations directly shape both the user interface and the behavior of the AI components.

6.1.3 Public Welfare

ScubaMind contributes to public welfare by making mental health-related behavioral insights more accessible through widely available consumer devices such as smartphones and wearables. Instead of requiring users to rely solely on expensive or infrequent clinical visits, the system provides a lower-barrier awareness tool that may help users better understand their own behavioral patterns over time.

This factor affected the design by encouraging accessibility, ease of use, and low-cost deployment. Since the system is designed to run on consumer mobile devices, it can potentially support a broader population and improve self-awareness in a practical and scalable manner.

6.1.4 Global Factors

Global factors affect the design mainly in terms of applicability across different user populations and technical environments. Mental health is a universal issue, but language resources, digital infrastructure, wearable ecosystems, and device capabilities vary significantly across regions.

In the current design, some AI and speech-related components are more feasible for English-centered processing and commonly available Android-based mobile environments. Therefore, while the overall problem is global, the current implementation is constrained by language and platform considerations. This influenced the design by encouraging a modular architecture that allows future adaptation for other languages, cultures, and technical ecosystems.

6.1.5 Cultural Factors

Cultural factors influence both the interpretation of behavioral signals and the way mental health-related outputs are perceived by users. Expressions of stress, emotional discomfort, help-seeking behavior, and even comfort with digital monitoring may differ across cultures.

For this reason, ScubaMind must avoid culturally rigid assumptions in both its analytical models and user-facing design. The system's wording, recommendations, and visual communication should remain neutral, respectful, and non-stigmatizing. This factor affected the design by reinforcing the need for careful model selection, cautious interpretation of behavioral data, and adaptable user communication strategies.

6.1.6 Social Factors

Social factors are highly influential in the design of ScubaMind, especially because mental health data is strongly associated with privacy concerns, stigma, and trust. Many individuals may feel uncomfortable using systems that appear intrusive or that transmit sensitive behavioral data to external servers.

This directly motivated the adoption of a **local-first architecture**. By collecting, storing, and analyzing data primarily on the user's device, the system reduces the feeling of surveillance and strengthens user trust. Social considerations also influenced the UI/UX design: the interface must be simple, calming, and non-intrusive so that users experiencing stress, anxiety, or low motivation can still use the system comfortably.

6.1.7 Environmental Factors

Environmental factors were considered mainly in terms of energy consumption, mobile efficiency, and long-term sustainable computation. Since ScubaMind uses continuous background sensing and on-device machine learning, there is a risk of unnecessary battery drain, device heating, and excess resource consumption if the system is not carefully optimized.

This factor influenced the design by requiring energy-efficient background services, optimized data collection intervals, and lightweight or quantized AI models where possible. In addition, unnecessary inference cycles should be minimized, and only the required amount of data should be stored locally. These decisions support both environmental responsibility and better mobile usability.

6.1.8 Economic Factors

Economic factors affect the system from both the development side and the end-user accessibility side. As a senior design project, the system must be developed with limited resources, which encourages the use of open-source tools, lightweight frameworks, and affordable integration strategies. In addition, the requirement for on-device AI may create a hardware barrier, since older or low-end smartphones may not be able to run all system components efficiently.

This factor influenced the design by prioritizing open-source technologies, modular implementation, and resource-aware engineering choices. Optional components such as advanced LLM-based support should also be treated carefully because they may introduce additional operational costs if external APIs are used in future versions.

Factor	Effect Level	Explanation
Public Health	Very High	The system directly addresses mental health awareness by monitoring behavioral patterns related to depression risk and providing early insights.
Public Safety	High	Since the system deals with sensitive behavioral data and mental health indicators, safety considerations such as non-diagnostic outputs and responsible recommendations are essential.
Public Welfare	High	The system aims to improve accessibility to behavioral self-awareness tools through widely available smartphones and wearable devices.
Global Factors	Medium	Mental health is a global problem; however, language support, device capabilities, and infrastructure differences limit universal applicability in the current implementation.
Cultural Factors	Low	Cultural differences may influence how users interpret behavioral insights and interact with mental health tools, but the system itself is largely culture-independent.
Social Factors	High	Privacy concerns, trust, and stigma around mental health significantly influence the system's design, especially the decision to adopt a local-first architecture.

Environmental Factors	Medium	Continuous sensing and on-device AI processing may affect device energy consumption, requiring efficient background services and optimized models.
Economic Factors	Medium	Development and deployment costs are influenced by hardware requirements, model efficiency, and potential use of external AI services in future versions.

Table 29 - Impact levels of various engineering design factors on the ScubaMind system

6.2 Standards

During the design and development of the ScubaMind system, several widely accepted international standards and best practices in software engineering and system design were considered. These standards help ensure that the system architecture, documentation, and implementation processes follow recognized engineering principles and maintain consistency, reliability, and maintainability.

6.2.1 IEEE Standards

The system design process follows principles inspired by standards defined by the Institute of Electrical and Electronics Engineers (IEEE). In particular, documentation and software lifecycle practices are aligned with general guidelines described in IEEE 12207 – Software Life Cycle Processes and IEEE 1016 – Software Design Description (SDD).

IEEE standards provide structured guidance for software architecture documentation, system decomposition, and interface definition. The structure of this design report, including architectural descriptions and subsystem documentation, follows these principles to ensure clarity and traceability between system requirements and design decisions.

6.2.2 ISO Standards

The ScubaMind system also considers software quality standards defined by the International Organization for Standardization (ISO). In particular, the design takes inspiration from the ISO/IEC 25010 Software Quality Model, which defines key quality characteristics such as reliability, usability, security, maintainability, and performance efficiency.

These quality attributes directly influenced the design goals of the system. For example, privacy-preserving local processing contributes to security, while the modular architecture of sensing, processing, and storage components improves maintainability and scalability.

6.2.3 UML Standards

Unified Modeling Language (UML) standards are used for visualizing the system architecture and subsystem interactions within the design documentation. UML diagrams provide a standardized way to represent system components, data flows, and interactions between modules.

In this project, UML-inspired diagrams such as system architecture diagrams, subsystem decomposition diagrams, and interaction diagrams are used to clearly illustrate the relationships between the mobile application layer, data collection services, AI processing modules, and storage components. These diagrams follow the general conventions defined by the UML specification maintained by the Object Management Group (OMG).

6.2.4 Object-Oriented Design (OOD)

The ScubaMind system is designed following object-oriented design principles that promote modularity, maintainability, and clear separation of responsibilities between system components. The use of object-oriented concepts allows the system to be structured into well-defined modules with clear responsibilities and interfaces.

Principles such as encapsulation, modularity, and separation of concerns influence the system architecture. For example, different components of the system, such as data collection services, AI processing modules, and user interface components, are designed as independent modules that interact through clearly defined interfaces. This approach simplifies system maintenance and allows future extensions or modifications without affecting the entire system.

7. Teamwork Details

The development of the ScubaMind system was carried out through a coordinated teamwork process in which responsibilities were distributed according to the technical expertise, experience, and interests of each team member. Since the project integrates multiple components such as mobile application development, passive sensing, artificial intelligence modeling, behavioral analysis, and system integration, the team adopted a structured approach to task distribution in order to ensure efficient development of different subsystems.

During the project, team members contributed to various components of the ScubaMind architecture while maintaining continuous coordination with each other. Regular communication between members allowed the team to synchronize the development of different modules such as sensing components, AI models, interaction modules, and system infrastructure. This ensured that individual subsystems evolved consistently with the overall architecture and system design goals.

In addition to technical development, the team collaboratively participated in architectural discussions, system design decisions, implementation planning, and integration processes.

The coordinated workflow allowed the team to address technical challenges collectively and ensured that the final system architecture remained coherent, scalable, and well-integrated.

7.1 Contributing and functioning effectively

Each team member contributed to the project by working on different subsystems of the ScubaMind architecture while maintaining close coordination with the rest of the team. Contributions also reflect the responsibilities that members previously assumed during earlier phases of the project.

Merve Güleç contributed to the behavioral data analysis and modeling components of the system. She worked on GPS-based behavioral data processing and developed machine learning models using the StudentLife dataset to analyze behavioral patterns related to mental well-being. Her work supported the activity recognition and behavioral modeling components of the system and contributed to the analytical layer that interprets passive sensing data.

Metin Çalışkan contributed to the speech analysis components of the system. His work focused on prosody-based analysis, extracting speech-related features that can provide behavioral insights related to emotional or cognitive states. This work contributes to the multimodal analysis capability of the ScubaMind system by enabling speech-based behavioral signals to complement other sensing modalities. In earlier project phases, he also contributed to the development of the NLP-related components of the system architecture.

Muhammed Fatih Başal focused on the infrastructure required for integrating the machine learning components into the overall system. He worked on establishing model pipelines and preparing the technical environment required for integrating machine learning models into the overall system architecture. His work ensured that analytical models could operate as part of a consistent processing pipeline. In earlier project phases, he also contributed to the design of the system's activity modeling components and helped shape the detailed system architecture.

Murathan Işık worked primarily on the mobile interaction layer of the system. In the later stages of development, he focused on the keyboard interaction module by adapting an externally sourced keyboard framework and modifying it to meet the requirements of the ScubaMind application. In addition, he worked on user interface improvements to enhance usability and overall user experience within the mobile application.

Yiğit Koşum contributed to the conversational AI functionality of the system. He worked on integrating the chat interface and implementing the LLM-based conversational interaction component that allows users to communicate with the system through an AI-supported chat assistant. This module forms an important part of the user interaction layer of the system. In earlier stages of the project, he also contributed to project coordination, system planning activities, and the development of the mobile application infrastructure.

By distributing responsibilities across different technical areas such as behavioral sensing, speech analysis, conversational AI, mobile interaction, and machine learning infrastructure,

the team ensured that multiple parts of the system architecture could be developed simultaneously while maintaining integration across modules.

7.2 Collaborative environment

Throughout the project, the team maintained a collaborative and inclusive working environment in which all members actively participated in discussions and decision-making processes. Weekly group meetings were conducted regularly to discuss project progress, evaluate technical challenges, and coordinate upcoming development tasks. Important architectural and implementation decisions were made collectively after consulting all members of the team.

Project documentation and reports were prepared collaboratively with contributions from all team members. Instead of assigning documentation tasks to specific individuals, the team worked together to review, improve, and finalize the reports in order to ensure consistency and accuracy across different sections of the documentation.

Similarly, the preparation of project presentations was carried out collectively. All members contributed to organizing presentation materials, improving explanations of the system architecture, and preparing visual content that clearly represents the functionality of the system.

The project demonstration was also presented with the participation of all team members. During the demonstration sessions, members supported each other in explaining different aspects of the system architecture and system functionality, ensuring that the design and implementation of the ScubaMind system were communicated clearly.

Through continuous communication, shared responsibilities, and collaborative decision-making, the team maintained a cooperative working environment that allowed members to coordinate their work effectively and support each other during the development process.

7.3 Leadership

Leadership responsibilities within the project were distributed across different subsystems and development stages. This distributed leadership approach allowed the team to manage the complexity of the system effectively and ensured that each major subsystem was guided by a team member with relevant expertise.

Merve Güleç took leadership roles in the behavioral modeling aspects of the project and guided the development of GPS-based behavioral analysis and activity modeling components of the system.

Metin Çalışkan led the development of the speech prosody analysis component and contributed to integrating speech-based behavioral signals into the multimodal analysis framework.

Muhammed Fatih Başal led the development of the machine learning infrastructure, focusing on establishing model pipelines and preparing the technical environment required for integrating machine learning models into the overall system architecture.

Murathan Işık led the development of the keyboard interaction module and worked on improving the user interface of the mobile application to enhance usability and interaction quality.

Yiğit Koşum took the lead in the development and integration of the conversational AI module, focusing on implementing the chat interface and integrating LLM-based interaction capabilities into the system.

By distributing leadership responsibilities across different technical components, the project ensured balanced management of system development while encouraging collaboration and knowledge sharing among team members.

8. Glossary

AES-256: Advanced Encryption Standard with a 256-bit key. The industry-standard symmetric encryption algorithm used by SQLCipher to secure the local database in ScubaMind.

Android Keystore System: A secure, hardware-backed storage environment (TEE/StrongBox) used to protect the dynamically generated cryptographic keys required for database encryption.

ASR (Automatic Speech Recognition): The technology and models used for transcribing spoken audio from journaling sessions into raw text for further analysis.

Background Service Orchestrator: A native Android component that manages the lifecycle of "Foreground Services" to ensure continuous, energy-efficient data acquisition without OS-level termination.

Behavioral Baseline: A user-specific reference profile constructed from historical behavioral data, used to interpret future deviations in the user's daily patterns.

Behavioral Insight: A system-generated interpretation of collected behavioral and physiological data, presented to the user to improve self-awareness rather than provide a clinical diagnosis.

BootReceiver: An Android broadcast receiver component responsible for restarting necessary background services (based on user privacy settings) when the device boots up.

DAO (Data Access Object): An interface utilized by the Room database library to abstract and manage SQL operations (e.g., UnifiedDataDao, VoiceRecordingDao).

Data Purge (Right to be Forgotten): A privacy mechanism that securely wipes the AppDatabase and resets EncryptedSharedPreferences, rendering all stored data cryptographically unrecoverable.

Decision-Support System: A software system designed to assist users in understanding patterns and making informed decisions, acting as an awareness tool rather than a diagnostic application.

Digital Phenotyping: The passive and continuous collection of behavioral and physiological signals from personal digital devices (smartphones, wearables) to analyze patterns related to mental well-being.

Embedded AI Core: The on-device machine learning component of ScubaMind, relying on TensorFlow Lite to analyze multimodal signals without transmitting sensitive data to the cloud.

EncryptedSharedPreferences: A secure Android storage mechanism used to safely store the dynamically generated UUID passphrase required to unlock the SQLCipher database.

FHIR (Fast Healthcare Interoperability Resources): A healthcare data standard used for representing and exchanging electronic health information in a structured and interoperable way.

Foreground Service: A specialized Android service class tied to a persistent notification, ensuring that critical data collection tasks (Step Counter, Sleep Tracker, Location Tracker) are not killed by the operating system.

Health Connect Bridge: A software integration layer that synchronizes physiological metrics (such as HRV and Sleep Stages) from Samsung Health and wearable devices into the local database.

HL7 (Health Level Seven): A family of international standards for the exchange, integration, sharing, and retrieval of electronic health information.

HRV (Heart Rate Variability): A physiological indicator collected from wearable devices representing the variation in time between heartbeats, often used as a biomarker for stress and autonomic nervous system balance.

IME (Input Method Editor): A specialized Android keyboard component custom-built for ScubaMind to capture raw text input across various apps, providing data for linguistic and sentiment analysis.

KVKK: The Turkish Personal Data Protection Law governing the processing, storage, and protection of personal data.

Late-Fusion Neural Network: A machine learning architecture used in ScubaMind's Multimodal Fusion unit, where separate data streams (phone behavior, text, prosody, wearables) are processed independently and synthesized at the final stage into a unified Risk Score.

LLM Companion: An advanced cloud-based module used to provide supportive, diary-style conversational guidance based on anonymized behavioral context.

Local-First Architecture: A core privacy-preserving design principle where all sensitive user data is collected, encrypted, and processed directly on the smartphone, minimizing cloud dependencies.

Method Channel: A communication bridge used to pass messages and data asynchronously between the Flutter UI codebase (Dart) and the Native Android backend (Kotlin).

MobilityDailyMetricsEntity / LocationSampleEntity: Specific Room database tables designed to store high-frequency GPS coordinates and the resulting derived spatial biomarkers.

Multimodal Fusion: The process of combining multiple distinct sources of behavioral and physiological data into a single, comprehensive analytical model.

NPU (Neural Processing Unit): Specialized hardware embedded in modern smartphones designed specifically to accelerate the execution of machine learning inference (such as TFLite models) efficiently.

On-Device Inference: The execution of trained machine learning models locally on the mobile device's processor, eliminating the need to send raw behavioral data to external servers.

PHQ-9 (Patient Health Questionnaire-9): A standard 9-item clinical depression questionnaire used in ScubaMind primarily as a baseline reference point and research label, rather than a direct diagnostic tool.

Prosody Analysis: The extraction and evaluation of acoustic features from voice recordings, such as pitch variance, speech rate, and intonation, serving as vocal biomarkers for emotional states.

Radius of Gyration: A computed mobility metric indicating the spatial variance of a user. It quantifies how far, on average, a user travels from their central or home location during a specific period.

Risk Score: A continuous or categorized behavioral indicator generated by the AI Module to reflect potential changes or trends in a user's mental well-being over time.

Room Persistence Library: An official Android Jetpack library that provides an abstraction layer over SQLite, allowing for robust, structured, and encrypted local database management.

Routine Index: A derived behavioral biomarker reflecting the regularity and predictability of a user's daily mobility traces and activities.

SQLCipher: An open-source extension to the SQLite database library that provides transparent, page-level 256-bit AES encryption to protect data at rest.

TFLite (TensorFlow Lite): A framework designed by Google for deploying lightweight machine learning models on mobile, edge, and IoT devices with low latency.

UnifiedDataEntity: The core analytical database table that aggregates and consolidates daily metrics (steps, sleep, PHQ scores, text fragments) into a single row per day.

UsageStatsManager API: A native Android interface utilized to track digital behavior patterns, including app usage duration and screen-on/screen-off events.

VoiceRecordingEntity: A database schema element that stores metadata for local audio journals, mapping file paths and durations to user-provided notes for prosody analysis

9. References

- [1] World Health Organization, *Depression*, World Health Organization, 2023. Available: <https://www.who.int/health-topics/depression>
- [2] R. C. Kessler and E. J. Bromet, "The epidemiology of depression across cultures," *Annual Review of Public Health*, vol. 34, pp. 119–138, 2013.
- [3] J. P. Onnela and S. Rauch, "Harnessing Smartphone-Based Digital Phenotyping to Enhance Behavioral and Mental Health," *Neuropsychopharmacology*, vol. 41, no. 7, pp. 1691–1696, 2016.
- [4] Wang, R., Chen, F., Chen, Z., Li, T., Harari, G., Tignor, S., Zhou, X., Ben-Zeev, D., & Campbell, A. T. StudentLife: Assessing Mental Health, Academic Performance and Behavioral Trends of College Students using Smartphones. Proceedings of the ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp), 2014. <https://doi.org/10.1145/2632048.2632054>
- [5] Moodpath GmbH. Moodpath – Interactive Mental Health Assessment Application. Available: <https://www.moodpath.de> Accessed: 2026. (Not: Moodpath daha sonra MindDoc olarak rebrand edildi ama Moodpath olarak referans vermek hâlâ kabul edilebilir.)
- [6] Mindstrong Health. Mindstrong – Digital Mental Health Platform. Available: <https://mindstronghealth.com> Accessed: 2026.
- [7] Apple Inc. Apple HealthKit Documentation. Available: <https://developer.apple.com/healthkit/> Google LLC. Google Fit – Health and Activity Tracking Platform. Available: <https://developers.google.com/fit>